En el mundo de la programación, la **creación de aplicaciones confiables y de alta calidad** es una prioridad constante. Sin embargo, este proceso puede ser desafiante y complejo, especialmente cuando se enfrenta a la necesidad de mantener un código limpio, funcional y libre de errores. En este contexto, entra en juego una poderosa herramienta: las pruebas unitarias.

Las **pruebas unitarias** son una inversión invaluable en el desarrollo de software, ya que no solo validan el correcto funcionamiento de cada unidad de código de manera aislada, sino que también aportan una serie de ventajas significativas que repercuten positivamente en todo el ciclo de desarrollo. En esta clase, exploramos los puntos fundamentales que hacen de los tests unitarios una estrategia esencial para ahorrar tiempo y esfuerzo a largo plazo, mientras se construye un software sólido y confiable.

Las **pruebas unitarias** consisten en verificar el funcionamiento individual de las piezas más pequeñas del código, asegurando que cada una de ellas trabaje correctamente antes de integrarlas en el conjunto más amplio del programa.

Un **test unitario** se enfoca en una unidad indivisible de código, como una función o método. El objetivo principal es verificar que la unidad funcione correctamente en diferentes escenarios. Mediante la simulación de entradas y la evaluación de resultados, podemos detectar errores antes de que afecten al sistema en su conjunto. Los **tests unitarios** deben ser rápidos, aislados y repetibles, lo que facilita la identificación y corrección de problemas.

Ventajas de las pruebas unitarias

Detección temprana de errores

Los tests unitarios permiten identificar rápidamente problemas y errores en el código, lo que facilita la corrección antes de que se propaguen a otras partes del programa.

Mejora la calidad del código:

Al escribir tests, los desarrolladores deben pensar en la funcionalidad y el diseño del código. Esto conduce a una mayor calidad y comprensión del software.

Facilita el debugging

Si surge un error en el futuro, los tests unitarios actúan como una herramienta de depuración y pueden ayudar a aislar y solucionar problemas más fácilmente.

Refactorización segura

Los tests unitarios garantizan que, incluso después de realizar cambios en el código, las funcionalidades existentes continúen funcionando correctamente, evitando regresiones.

Fomenta el trabajo colaborativo:

Los tests unitarios facilitan la colaboración entre desarrolladores, ya que proporcionan una especificación clara y concreta de la funcionalidad esperada, permitiendo que varios desarrolladores trabajen en el mismo código de manera segura.

Las pruebas unitarias nos permiten detectar errores de forma temprana, lo que evita que se conviertan en problemas mayores más adelante. También mejoran la

calidad del código, al incentivar prácticas de desarrollo más sólidas y estructuradas.
Además, facilitan el proceso de depuración o "**debugging**", al proporcionar un entorno controlado para verificar el comportamiento de cada unidad de código y aislar errores de manera eficiente.
Finalmente, los tests unitarios permiten refactorizaciones seguras y promueven un trabajo colaborativo más fluido, al brindar la confianza necesaria para realizar cambios en el código sin temor a romper su funcionalidad.

La librería de test para Java: Junit

Empecemos con Junit5 Jupiter: una poderosa librería de pruebas unitarias para Java

Junit5 Jupiter es una potente librería de pruebas unitarias para Java, diseñada para facilitar el proceso de escribir y ejecutar tests de manera eficiente y efectiva. Su objetivo principal es proporcionar un marco de trabajo moderno y flexible que permita a los desarrolladores realizar pruebas unitarias de manera más sencilla y efectiva, mejorando así la calidad y confiabilidad del software desarrollado.

Componentes de Junit5 Jupiter

Anotaciones

Cuenta con un conjunto de anotaciones que permiten definir y configurar las pruebas unitarias de manera declarativa. Algunas de las anotaciones más comunes son @Test, que indica que un método es una prueba unitaria, y @BeforeEach y @AfterEach, que permiten configurar acciones que se ejecutan antes y después de cada prueba.

Asunciones

Incluye un conjunto de métodos de asunciones que permiten verificar supuestos sobre el entorno en el que se ejecutan las pruebas. Por ejemplo, la asunción assumeTrue() permite omitir una prueba si cierta condición no se cumple, lo que es útil para evitar errores innecesarios.

Assertions

La librería ofrece una amplia gama de métodos de aserción que permiten verificar los resultados esperados de las pruebas. Estas aserciones permiten comprobar si un resultado es verdadero, falso, nulo, igual, etc., lo que facilita la validación de los resultados de las pruebas.

Extensiones

Permite la creación de extensiones personalizadas que agregan funcionalidades adicionales a las pruebas. Estas extensiones pueden utilizarse para configurar el entorno de prueba, capturar información de ejecución y muchas otras funcionalidades.

Integración con otros frameworks

Se integra de manera nativa con otros frameworks y librerías populares, lo que facilita su uso y permite aprovechar sus características junto con otras herramientas de testing.

Junit5 Jupiter se ha convertido en una opción preferida para realizar pruebas unitarias en proyectos Java debido a su facilidad de uso y la amplia variedad de funcionalidades que ofrece.

El test unitario es una técnica fundamental dentro del desarrollo de software que permite verificar el correcto funcionamiento de unidades individuales de código, como métodos o funciones, de manera aislada y sin depender de otros componentes del sistema. Su objetivo principal es asegurar que cada unidad de código, por sí sola, produce los resultados esperados y cumple con los requerimientos establecidos.

Características del Test Unitario

Aislamiento de la Unidad: En un test unitario, la unidad de código que se está probando se ejecuta de forma aislada, sin interactuar con otras partes del sistema. Para lograr esto, se utilizan técnicas como el uso de mocks o stubs para simular las dependencias externas y asegurar que los resultados obtenidos son consistentes y predecibles

Independencia de Entorno: Los test unitarios se diseñan de manera que sean independientes del entorno en el que se ejecutan. Esto significa que no deben depender de bases de datos, servicios externos, o cualquier otro recurso que pueda variar entre diferentes entornos de pruebas o máquinas de desarrollo.

Cobertura Específica: Cada test unitario se enfoca en una única unidad de código y verifica una funcionalidad específica. Esto permite identificar rápidamente la causa de un fallo en caso de que se produzca, ya que se sabe exactamente qué parte del código está siendo probada .

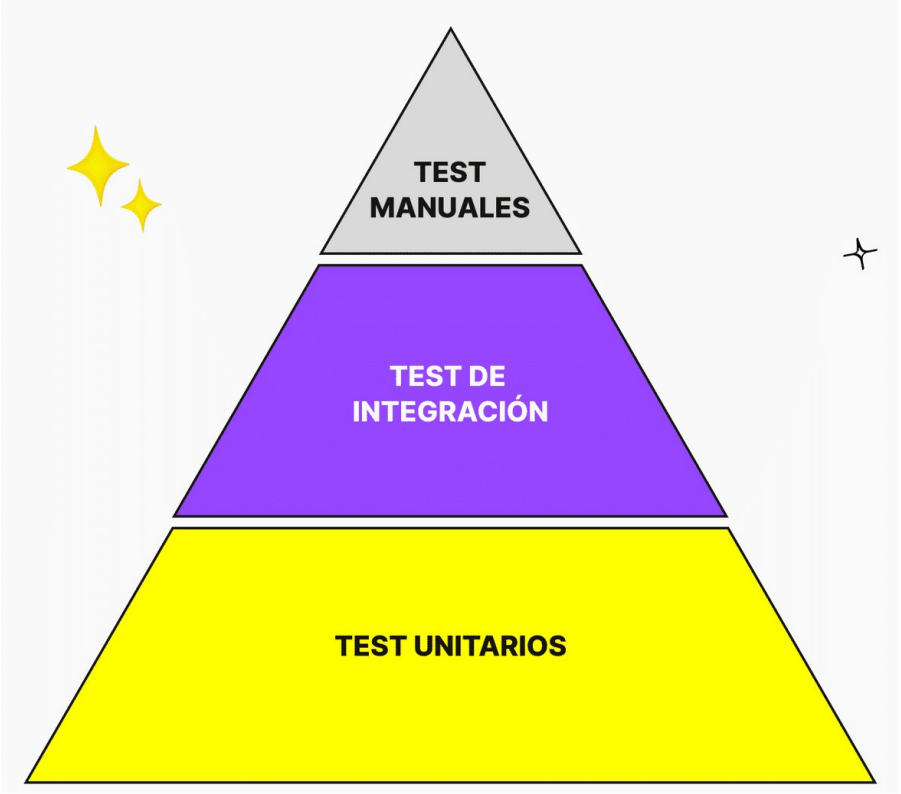
Ejecución Rápida: Los test unitarios están diseñados para ser rápidos y eficientes en su ejecución. Esto permite realizar pruebas de manera frecuente y ágil durante el desarrollo, lo que facilita la detección temprana de errores y la iteración en el proceso de diseño e implementación .

Automatización: Los test unitarios son automatizados, lo que significa que pueden ser ejecutados de forma repetida y sistemática cada vez que se realicen cambios en el código. Esta automatización asegura que las pruebas sean consistentes y repetibles, facilitando la detección de regresiones y cambios inesperados en el comportamiento del código

Evaluación de Resultados: La evaluación de los resultados de los test unitarios es simple y clara. Cada test debe definir claramente el resultado esperado y compararlos con los resultados obtenidos. Si los resultados coinciden, el test se considera exitoso. En caso contrario, se detecta un error que debe ser corregido y se debe repetir la ejecución del test para verificar la corrección

¿Cómo está formado un test unitario?

La composición de los test unitarios utilizando la práctica GIVEN/WHEN/THEN es una técnica de escritura de pruebas que ayuda a mejorar la claridad y organización de los casos de prueba. Esta metodología se basa en dividir el caso de prueba en tres secciones distintas: el contexto (GIVEN), la acción (WHEN) y el resultado esperado (THEN). Cada componente fundamental juega un papel importante en esta estructura:



@DisplayName: Esta anotación se utiliza para proporcionar un nombre descriptivo y significativo a cada caso de prueba. Ayuda a identificar claramente qué funcionalidad o comportamiento está siendo probado y mejora la legibilidad de los resultados de las pruebas. Un buen nombre de prueba debe ser informativo y expresar la intención del caso de prueba.

@Test: La anotación @Test se utiliza para marcar un método como un caso de prueba. Esto indica al entorno de pruebas que el método debe ser ejecutado como parte de las pruebas unitarias. Cada método marcado con @Test representa un caso de prueba individual y debe estar aislado y no depender de otros métodos de prueba

@Disabled: La anotación @Disabled se utiliza para desactivar temporalmente un caso de prueba. Puede ser útil cuando se está desarrollando una funcionalidad y se necesita excluir ciertos casos de prueba que aún no están completos o están fallando. Esto permite centrarse en la resolución de problemas en otras áreas y evitar la distracción de casos de prueba incompletos

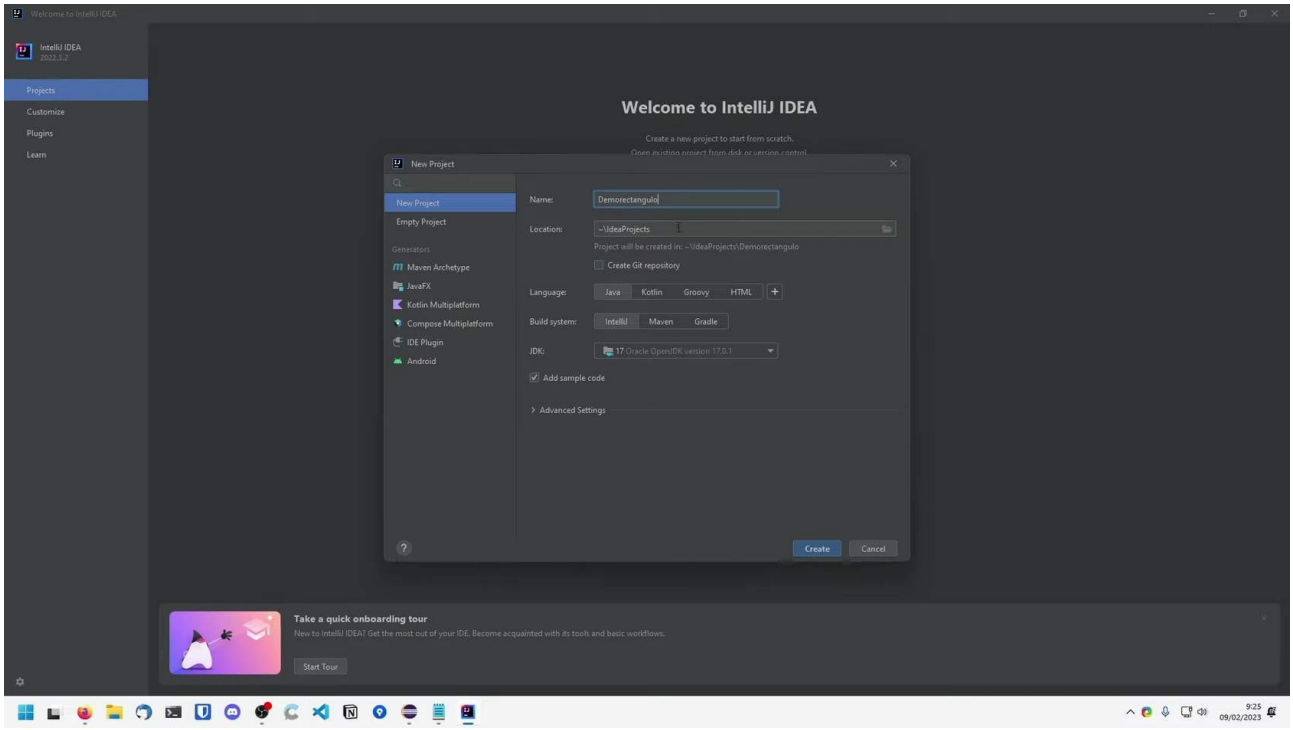
La estructura GIVEN/WHEN/THEN guía la organización del código del caso de prueba de la siguiente manera:

GIVEN (Contexto): En esta sección, se establece el estado inicial o contexto necesario para el caso de prueba. Esto puede incluir la creación de objetos, la configuración de variables, la preparación de datos y cualquier otra configuración necesaria para realizar la prueba

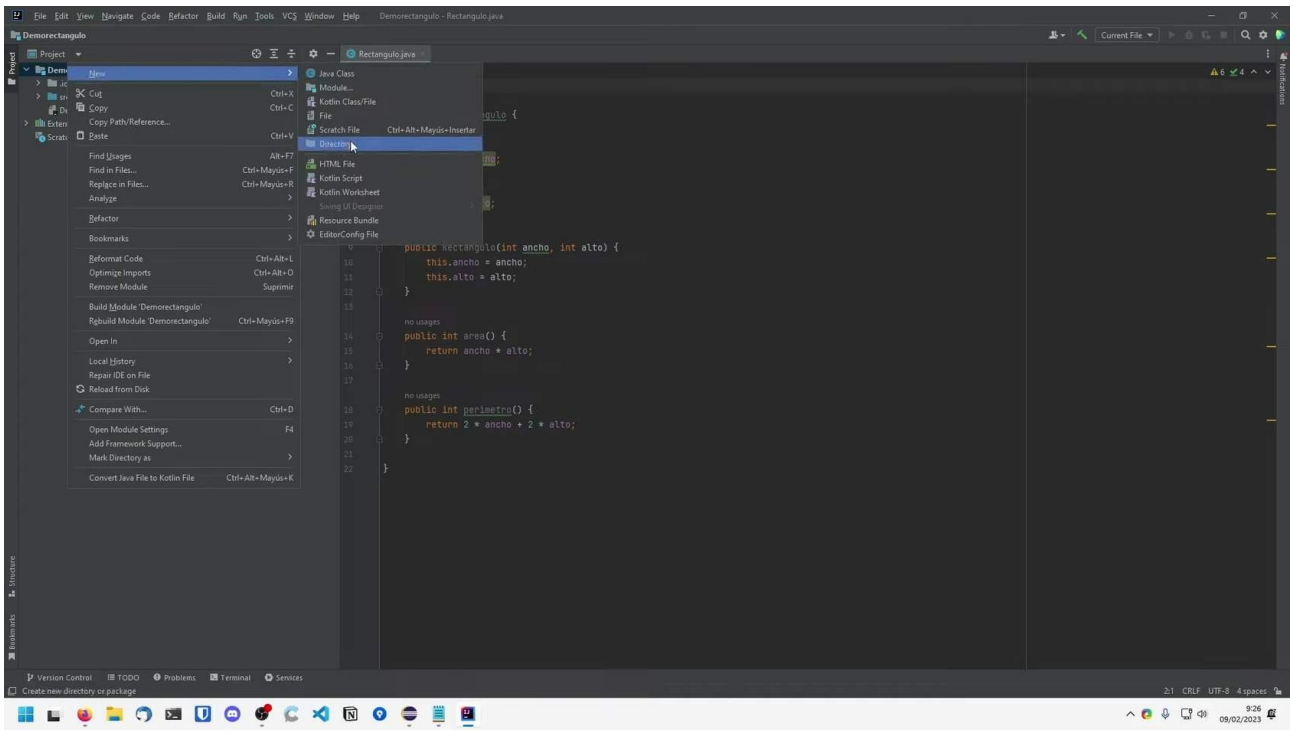
WHEN (Acción): En esta sección, se lleva a cabo la acción o el evento que se está probando. Es decir, se ejecuta el método o función que se quiere probar, pasando los datos necesarios que se prepararon en la sección GIVEN.
THEN (Resultado Esperado): En esta sección, se verifica si el resultado de la acción es el esperado. Se utilizan aserciones (assertions) para comparar el resultado obtenido con el resultado esperado. Si las aserciones son verdaderas, el caso de prueba se considera exitoso. Si alguna aserción falla, indica que el comportamiento del código no es el esperado y se debe investigar y corregir el problema.

¿Y cómo escribimos un test unitario?

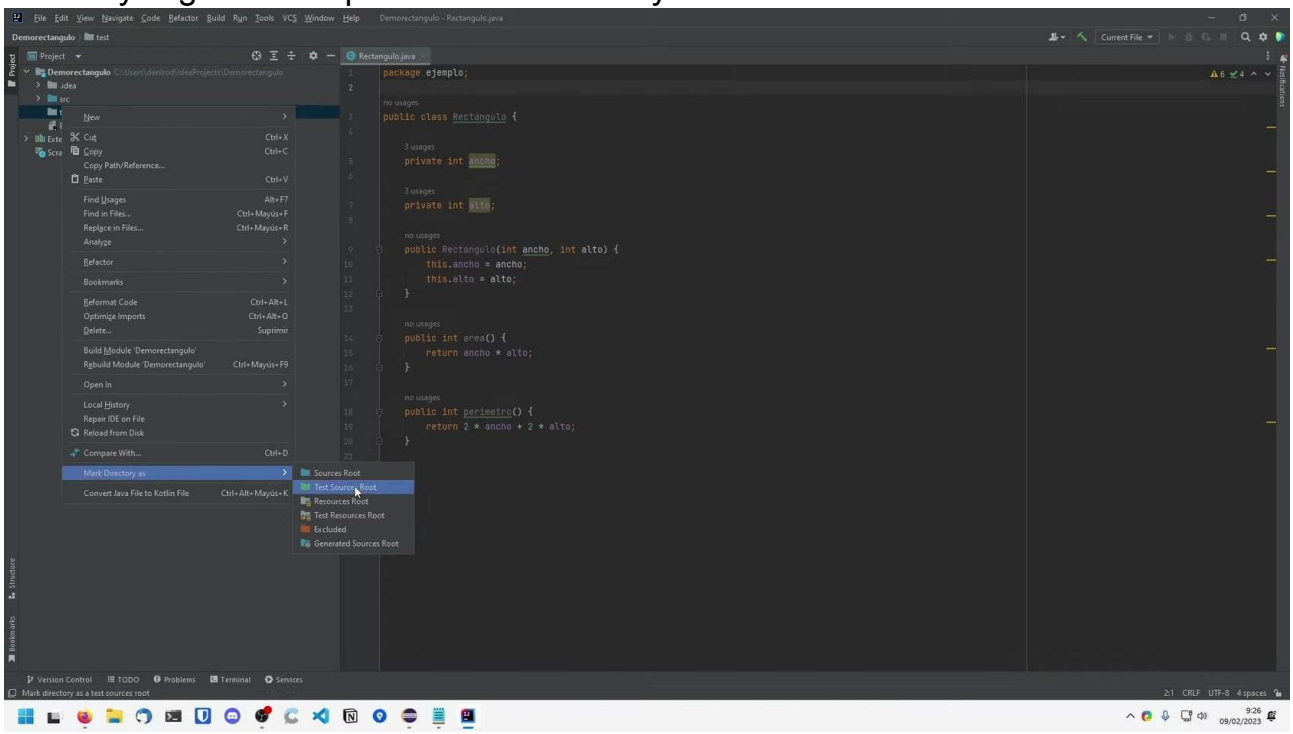
A continuación, repasemos brevemente cómo crear un test unitario y otra forma de poder importar la librería de Junit 5 a nuestra aplicación:
Creamos un proyecto.



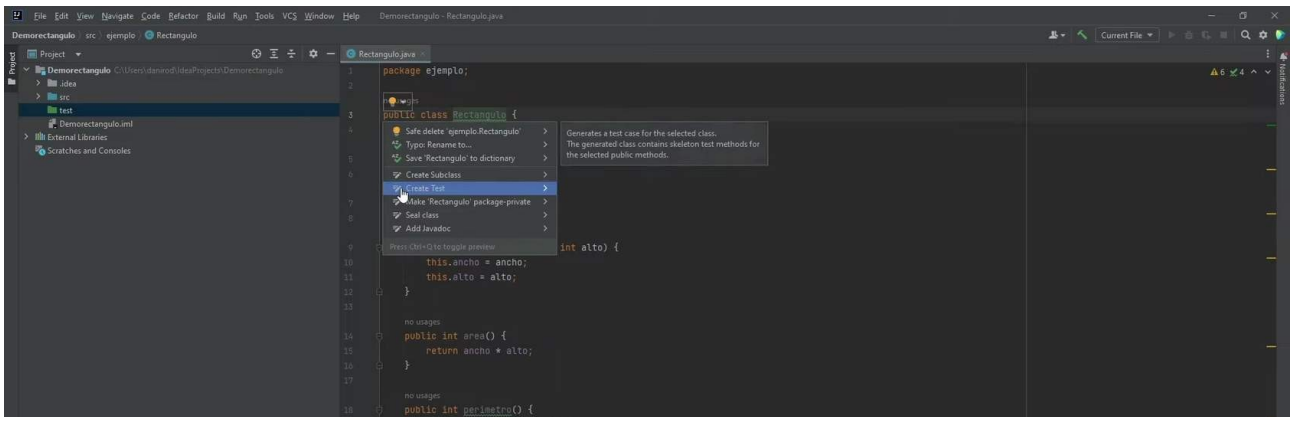
Creamos un directorio que se llame test, a la misma altura que el directorio src



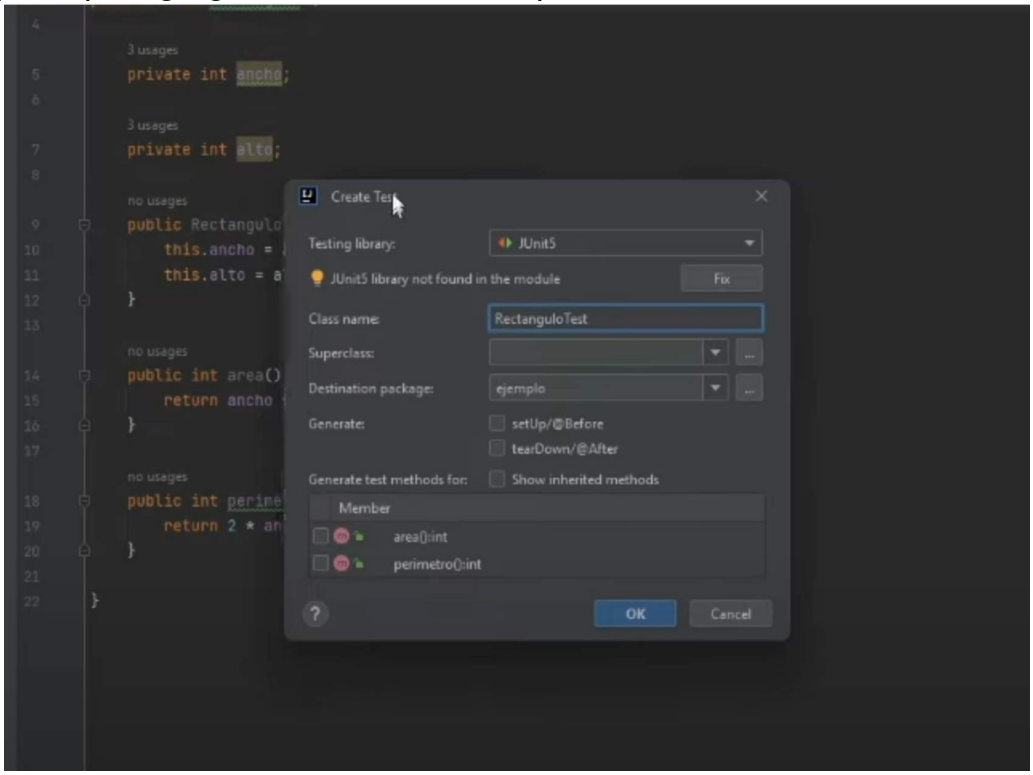
La marcamos como ruta de tests, haciendo click derecho en el directorio recién creado y eligiendo la opción “Mark Directory as”



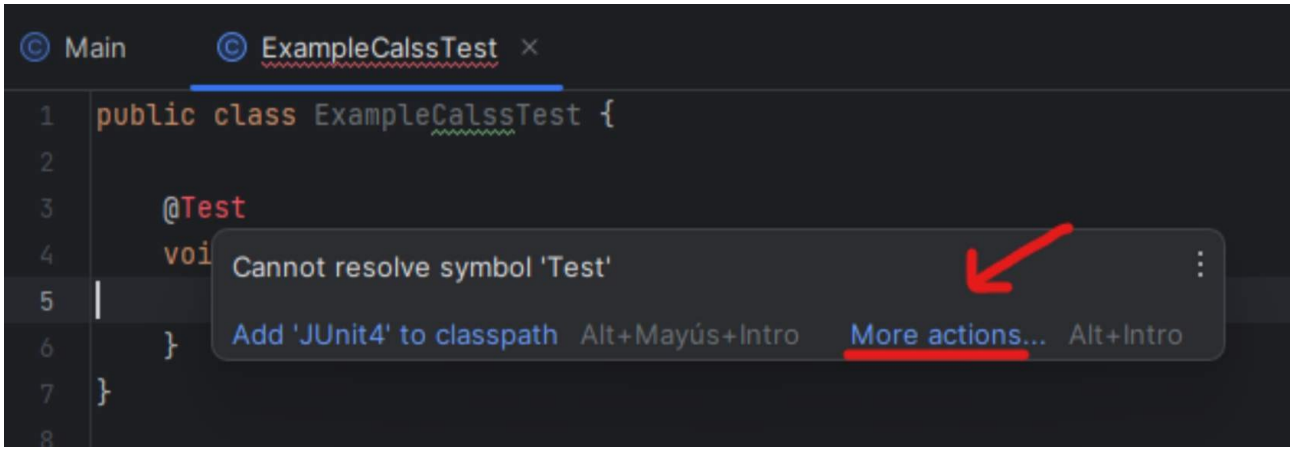
Vamos a nuestra clase y ponemos “crear test”

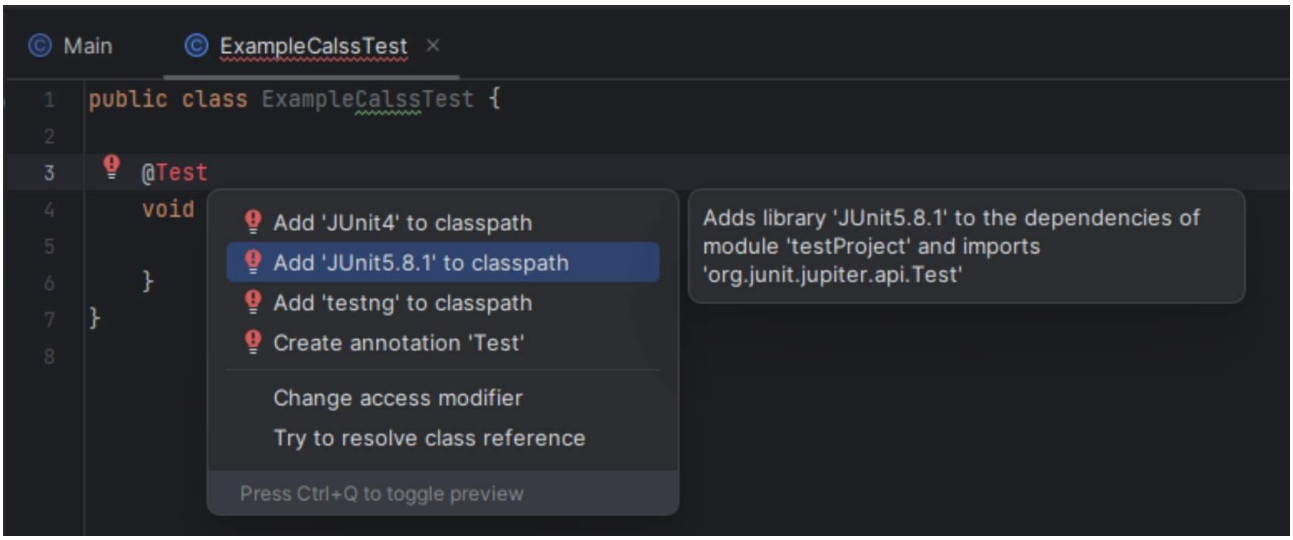


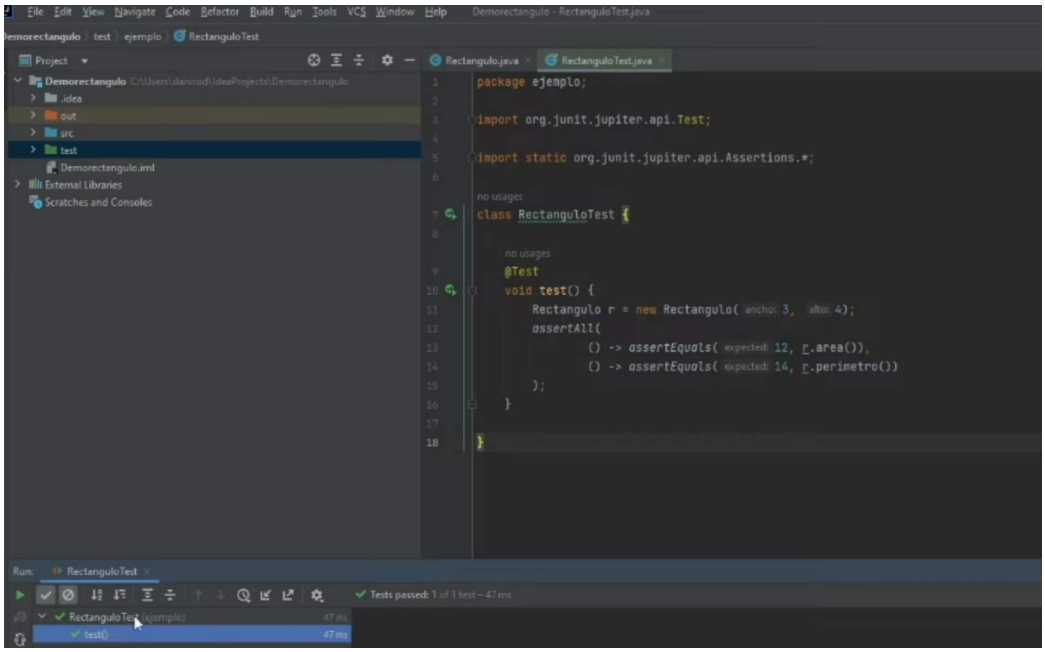
Usamos el botón de “fix” para que descargue la librería a usar. En caso de que el botón no esté, podemos posarnos sobre alguna anotación como @Test y ayudarnos del IDE para que agregue la librería al classpath



Tenemos la clase para nuestro test







De esta manera lo que hacemos es configurar el ambiente necesario para poder levantar nuestros tests

Características del Test de Integración

Los tests de integración son pruebas que evalúan la interacción entre diferentes componentes o módulos del sistema para verificar si funcionan correctamente en conjunto. A diferencia de los tests unitarios, que se centran en aislar unidades individuales de código, los tests de integración evalúan cómo las diferentes partes del sistema interactúan y colaboran entre sí para cumplir con las funcionalidades requeridas. Estas pruebas son más complejas y pueden involucrar bases de datos, servicios externos o cualquier otro recurso compartido.

Objetivo del Test de Integración:

El principal objetivo de los tests de integración es asegurar que los diferentes componentes del sistema trabajen adecuadamente en conjunto y cumplan con los

requisitos de integración establecidos. Estas pruebas son esenciales para verificar la interoperabilidad entre los módulos y garantizar que todas las partes se integren de manera efectiva. Además, los tests de integración permiten detectar posibles problemas de comunicación y colaboración entre los componentes, lo que contribuye a la creación de un software confiable y robusto.

Desarrollo del Test de Integración:

Para realizar un test de integración, se sigue una estructura similar a los tests unitarios, aunque con un alcance más amplio para abarcar la interacción entre múltiples componentes. A continuación, se detallan los pasos clave en el desarrollo de un test de integración:

Configuración del escenario (GIVEN): En esta etapa, se prepara el entorno y se configuran los componentes necesarios para la prueba de integración. Esto puede implicar la inicialización de bases de datos, configuración de servicios externos o cualquier otro elemento requerido para la interacción entre los módulos.

Ejecución de la interacción (WHEN): En esta sección, se realiza la acción que se quiere probar, involucrando la comunicación y colaboración entre los componentes. Es el momento de ejecutar el código que se busca probar en el contexto de integración.

Verificación del resultado (THEN): En esta fase, se evalúa si el resultado de la interacción entre los componentes es el esperado. Se comprueba si se cumplen los requisitos de la funcionalidad que se está probando y si los resultados obtenidos son consistentes con las expectativas.

Imaginemos que estamos desarrollando un sistema de compras en línea y queremos realizar un test de integración para verificar que la funcionalidad de agregar un producto al carrito de compras funciona correctamente. Para esto, consideraremos tres componentes principales: CarritoDeCompras, Producto y BaseDeDatos.

Configuración del escenario (GIVEN): Preparamos el escenario para la prueba definiendo algunos productos y un carrito de compras vacío. Además, creamos una base de datos en memoria que almacenará los productos y el carrito de compras

Ejecución de la interacción (WHEN): Agregamos algunos productos al carrito de compras y luego verificamos que la cantidad de productos en el carrito sea correcta.

Verificación del resultado (THEN): Comprobamos que los productos se hayan agregado correctamente al carrito y que la cantidad de productos en el carrito sea la esperada.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.List;

public class CarritoDeComprasIntegrationTest {

    private CarritoDeCompras carrito;
    private BaseDeDatos baseDeDatos;

    @BeforeEach
    public void setUp() {
        // Configuración de la base de datos en memoria
        baseDeDatos = new BaseDeDatosEnMemoria();
```

```
        // Creación de productos en la base de datos
        Producto producto1 = new Producto("123", "Laptop", 1000);
        Producto producto2 = new Producto("456", "Teléfono", 500);
        baseDeDatos.guardarProducto(producto1);
        baseDeDatos.guardarProducto(producto2);

        // Creación de un carrito de compras vacío
        carrito = new CarritoDeCompras();
    }

    @Test
    public void testAgregarProductosAlCarrito() {
        // Agregar productos al carrito
        carrito.agregarProducto(baseDeDatos.obtenerProductoPorCodigo("123"));
        carrito.agregarProducto(baseDeDatos.obtenerProductoPorCodigo("456"));

        // Verificar que los productos se agregaron correctamente
        List<Producto> productosEnCarrito = carrito.getProductos();
        assert productosEnCarrito.size() == 2;
                                                                    assert
productosEnCarrito.contains(baseDeDatos.obtenerProductoPorCodigo("123"));
                                                                    assert
productosEnCarrito.contains(baseDeDatos.obtenerProductoPorCodigo("456"));
    }
}
```

En este ejemplo, utilizamos @BeforeEach para configurar la base de datos y crear algunos productos antes de cada test. Luego, en el test testAgregarProductosAlCarrito, interactuamos con el carrito de compras y la base de datos para agregar productos al carrito y verificar que se hayan agregado correctamente.

Este ejemplo muestra cómo los diferentes componentes del sistema (CarritoDeCompras, Producto y BaseDeDatos) interactúan entre sí para llevar a cabo la funcionalidad de agregar productos al carrito de compras. La prueba de integración nos permite asegurarnos de que todos estos componentes funcionen adecuadamente en conjunto y que la funcionalidad del sistema sea coherente y confiable.

Test unitarios vs tests de integración

Aunque los tests de integración y los tests unitarios comparten la estructura GIVEN/WHEN/THEN y ambos buscan mejorar la calidad del software, se diferencian en su enfoque. Los tests de integración evalúan la colaboración y comunicación entre los componentes, mientras que los tests unitarios se concentran en la funcionalidad específica de unidades individuales de código.

Los tests de integración son más complejos y pueden requerir la configuración de recursos compartidos, como bases de datos, mientras que los tests unitarios son más aislados y no involucran la interacción con otros componentes. Ambos tipos de pruebas son complementarios y juntos contribuyen a una estrategia completa de testing para asegurar la calidad y fiabilidad del software

Características	Test Unitario	Test de Integración
Objetivo	Probar unidades de código de forma aislada	Probar la integración entre diferentes componentes
Aislamiento	Se aislan del resto del sistema y se prueban individualmente	Requieren la colaboración de varios componentes
Mocking	Pueden usar mocks para simular dependencias externas	No utilizan mocks ya que involucran componentes reales
Alcance	Prueban una función, método o clase específica	Prueban la interacción entre varias funciones o módulos
Dependencias	Se prueban sin dependencias externas	Requieren la presencia de componentes reales
Ejecución	Son rápidos y se ejecutan con frecuencia	Son más lentos debido a la necesidad de integración
Tamaño de la Prueba	Pueden ser pequeñas y rápidas de escribir	Pueden ser más complejas y llevar más tiempo
Requerimientos	No requieren un entorno específico o base de datos	Pueden requerir un entorno completo para la integración
Cobertura de Prueba	Cubren un área específica del código	Cubren varias partes del sistema en conjunto

Es importante destacar que tanto los tests unitarios como los tests de integración son fundamentales en el proceso de desarrollo de software. Los tests unitarios nos permiten probar unidades individuales de código para asegurarnos de que funcionen correctamente de manera aislada. Por otro lado, los tests de integración nos permiten verificar que los diferentes componentes del sistema trabajen adecuadamente juntos, asegurando la correcta interacción entre ellos.

¡Sumamos a Mockito!

Mockito es una popular biblioteca de pruebas en Java que permite crear objetos simulados o "mocks" para realizar pruebas unitarias más eficientes y efectivas. Estos mocks simulan el comportamiento de objetos reales y se utilizan para aislar y probar unidades individuales de código de manera independiente

Principales características de Mockito

- **Creación de mocks:** Mockito permite crear fácilmente objetos simulados de interfaces o clases con un comportamiento predefinido. Esto permite aislar las dependencias externas y centrarse en la unidad de código que se está probando
- **Configuración del comportamiento:** Los mocks pueden ser configurados para devolver valores específicos, lanzar excepciones o realizar otras acciones cuando se llaman a sus métodos. Esto permite simular diferentes escenarios de prueba.
- **Verificación de interacciones:** Mockito proporciona métodos para verificar si ciertos métodos de los mocks fueron llamados o no, y con qué

argumentos. Esto es útil para asegurarse de que el código bajo prueba interactúa correctamente con sus dependencias.

- **Sintaxis clara y legible:** Mockito ofrece una sintaxis simple y legible que facilita la escritura y comprensión de las pruebas unitarias. Esto hace que el código de prueba sea más mantenible y fácil de leer.
  Integración con frameworks de pruebas: Mockito se integra bien con otros frameworks de pruebas, como JUnit, facilitando su uso en proyectos existentes

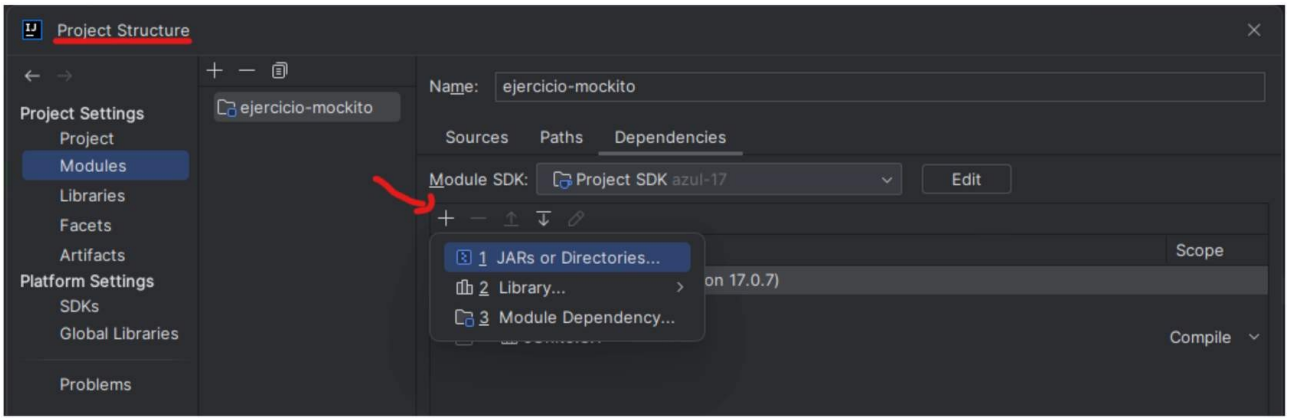
  En resumen, Mockito es una herramienta poderosa y versátil que simplifica la creación de pruebas unitarias en Java y ayuda a garantizar la calidad y robustez del código.

Para poder ver más acerca de esto te invitamos a ver este video https://youtu.be/j9k3epjUgr8

¿Y cómo configuramos Mockito en nuestra aplicación?

Debido a que estamos generando aplicaciones de Java sin gestores de dependencias (como Maven o Gradle), trabajar con Mockito es un poco más complejo que descargar un solo JAR, ya que el mismo proyecto de Mockito usa otras dependencias como objenesis o byte-buddy.
Podrían navegar en la web, y hacer el pequeño trabajo de investigar las librerías necesarias para hacer que Mockito funcione, pero para ahorrarnos ese pequeño paso, les dejamos a continuación el compendio de JAR's que deben agregar al classpath.
Compendio de JAR's Deben de agregar al classpath.

https://drive.google.com/drive/folders/1SSeQyOa3LtdCe11nJdEK0LZSVAGlQUqi?usp=share_link
Una vez descargado el comprimido con todos los JAR's, vamos a nuestra aplicación de Java e ingresamos en File -> Project Structure, y con el botón +, agregamos todos los JAR's al classpath. Damos en aplicar y ¡listo! Ya tenemos Mockito listo para usar.



Veamos las anotaciones básicas que podemos usar en nuestros tests unitarios para mejorar las funcionalidades y aserciones de los mismos, junto con algunos ejemplos básicos de sintaxis y contexto para poder usarlas:

@Mock: se utiliza para crear objetos simulados (mocks) de interfaces o clases. Estos mocks se utilizan para simular el comportamiento de las dependencias externas durante las pruebas unitarias.

```
import org.mockito.Mock;
public class MyTest {
    @Mock
    private EjemploDependencia dependencia;

    // ... código de prueba ...
}
```

@InjectMocks: se utiliza para inyectar automáticamente los mocks creados con la anotación @Mock en la instancia de la clase bajo prueba. Esto evita la necesidad de crear manualmente los mocks e insertarlos en la clase.

```
import org.mockito.InjectMocks;
import org.mockito.Mock;
public class MiTest {
    @Mock
    private EjemploDependencia dependencia;;

    @InjectMocks
    private MiClaseTesteada miClaseTesteada;

    // ... código de prueba ...
}
```

@Spy: se utiliza para crear objetos espía, que son objetos reales con ciertos métodos simulados. A diferencia de los mocks, los espías mantienen el comportamiento original de los métodos no simulados, lo que permite realizar pruebas en objetos reales con algunos métodos simulados.

```
import org.mockito.Spy;
public class MiTest {
    @Spy
    private ObjetoReal objetoReal;
    // ... código de prueba ...
}
```

Uso de métodos principales:

when/thenReturn: El método when() se utiliza para especificar el comportamiento de un mock cuando se llama a uno de sus métodos. Luego, el método thenReturn() se utiliza para indicar el valor que debe devolver el mock cuando se llame al método especificado.

```
import static org.mockito.Mockito.*;
public class MiTest {
    @Test
    public void testComportamientoMock() {
        MiDependencia miDependenciaMock = mock(MiDependencia.class);
        when(miDependenciaMock.getValue()).thenReturn(42);
        // ... código de prueba ...
    }
}
```

doReturn/when: En algunos casos, cuando el método a simular tiene una firma que no puede usarse con when/thenReturn, se puede utilizar doReturn() en combinación con el método when() para configurar el comportamiento del mock.

```
import static org.mockito.Mockito.*;
public class MiTest {
    @Test
    public void testComportamientoMock() {
        MiDependencia miDependenciaMock = mock(MiDependencia.class);
        doReturn(42).when(miDependenciaMock).getValue();
        // ... código de prueba ...
    }
}
```

verify: El método verify() se utiliza para verificar si se ha llamado a un método específico de un mock y con qué argumentos. Esto permite comprobar si el código bajo prueba interactuó correctamente con sus dependencias.

```
import static org.mockito.Mockito.*;
public class MiTest {
    @Test
    public void testInteraccionClasePrueba() {
        MiDependencia miDependenciaMock = mock(MiDependencia.class);
                            ClasePruebaTesteada    clasePruebaTesteada    =    new
ClasePruebaTesteada(miDependenciaMock);
        // ... código de prueba ...
        verify(miDependenciaMock).métodoAEjecutar(arg1, arg2);
    }
}
```

doThrow: El método doThrow() se utiliza para simular que un método arroja una excepción. Se puede combinar con el método when() para configurar el comportamiento del mock y lanzar una excepción cuando se llama al método especificado

```
import static org.mockito.Mockito.*;
public class MiTest {
    @Test(expected = UnaExcepcionCualquiera.class)
    public void testDeExcepcion() {
        MiDependencia miDependenciaMock = mock(MiDependencia.class);
                           when(miDependenciaMock.metodoAEjecutar()).thenThrow(new
UnaExcepcionCualquiera());
        // ... código de prueba ...
    }
}
```

Estas anotaciones y métodos son fundamentales en Mockito para crear mocks, configurar su comportamiento y verificar las interacciones durante las pruebas unitarias. Con su uso adecuado, Mockito facilita la escritura de pruebas unitarias más efectivas y eficientes en Java.

En la siguiente página vas a encontrar un tutorial introductorio al testing.

Interacciones básicas con objetos Mock

¿Qué son los ArgumentMatchers?

Los ArgumentMatchers son una característica importante de la biblioteca de pruebas Mockito. Se utilizan para especificar condiciones y expectativas en los métodos

simulados o mockeados que se están probando. Es decir, ayudan a hacer coincidir y verificar los argumentos pasados a los métodos simulados durante las pruebas.
Estos ArgumentMatchers (o como el mismo nombre lo indica, los comparadores de argumentos) se utilizan principalmente en combinación con el método `when` para definir comportamientos específicos de los mocks. Los ArgumentMatchers permiten simular diferentes casos de entrada y verificar si los métodos simulados se comportan correctamente bajo esas condiciones.

¿Y cómo se pueden usar?

Es importante aclarar que los ArgumentMatchers se encuentran en la clase `org.mockito.ArgumentMatchers`, para poder verificar cómo y de dónde los importamos al momento de usarlos. Aquí hay algunos ejemplos de cómo se pueden utilizar:

Igualdad Simple

Puedes usar `eq` para verificar si un argumento es igual a otro:

```
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.*;
// ...
when(mockObject.method(eq("someValue"))).thenReturn(someResult);
```

Cualquier Valor

Puedes usar `anyString` para cualquier valor de un tipo String:

```
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;
// …
when(mockObject.method(anyString())).thenReturn(someResult);
```

Verificación de Argumentos

También puedes verificar que un método simulado se haya llamado con argumentos específicos:

```
import static org.mockito.Mockito.*;
// ...
verify(mockObject).method(eq("expectedValue"), anyInt());
```

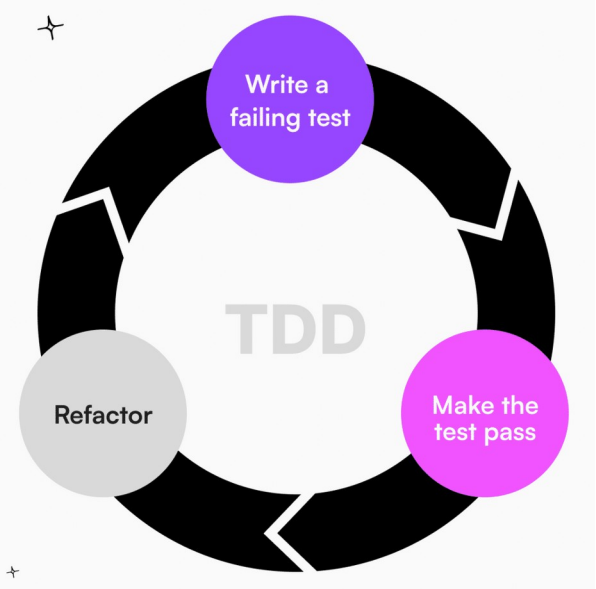
Los ArgumentMatchers son una herramienta poderosa en Mockito para especificar expectativas y verificar interacciones con métodos simulados. Al combinar ArgumentMatchers con los métodos `when` y `verify`, puedes crear pruebas más flexibles y sólidas al verificar el comportamiento de los objetos simulados bajo diferentes condiciones.
Antes de terminar esta lección, es importante que revisemos un concepto muy importante, con el cual seguramente nos toparemos en más de un mensaje de error, mientras estemos usando Mockito… el **stubbing**

En el contexto de Mockito, el *stubbing* se refiere a la definición del comportamiento de un mock cuando se llama a uno de sus métodos durante una prueba unitaria. Es decir, se configura el mock para que devuelva un valor específico, lance una excepción o realice alguna otra acción cuando se llame a un método en particular.
El stubbing se realiza utilizando los métodos *when/thenReturn* o *doReturn/when* combinados con los mocks. Por ejemplo:

```
// Crear el mock
List<String> mockList = mock(List.class);
// Definir el comportamiento del mock utilizando when/thenReturn
when(mockList.get(0)).thenReturn("Ejemplo de stubbing");
// Ejecutar el método bajo prueba
String result = mockList.get(0);
// Verificar el resultado del stubbing
assertEquals("Ejemplo de stubbing", result);
```

En este ejemplo, se crea un **mock** de la clase **List** y se define que cuando se llame al método get(0) del mock, debe devolver la cadena "Ejemplo de stubbing". Luego, se ejecuta el método get(0) del mock y se verifica que devuelva el valor esperado.
El **stubbing** es una parte fundamental de las pruebas unitarias, ya que permite controlar el comportamiento de las dependencias durante las pruebas y garantizar que el código bajo prueba funcione correctamente en diferentes escenarios.
Podes encontrar más acerca de esto en este video: https://youtu.be/xXO8ft-tsrY

¿Qué es el TDD?

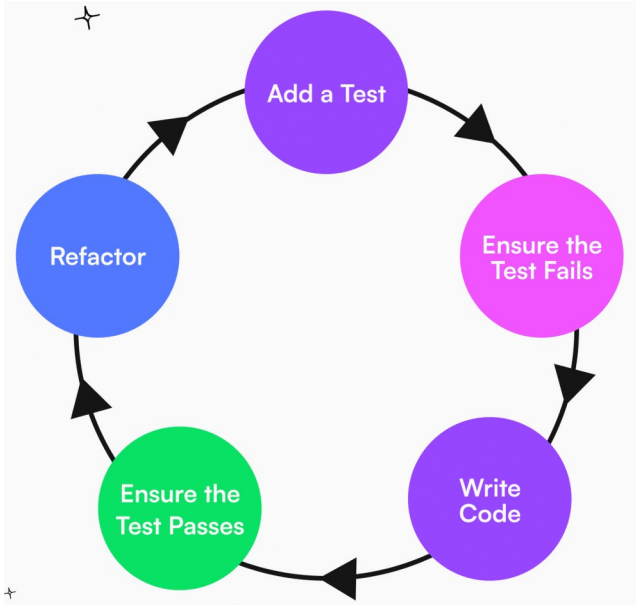


El Desarrollo Dirigido por Pruebas, o TDD por sus siglas en inglés (Test-Driven Development), es una práctica de desarrollo de software que enfatiza escribir pruebas antes de escribir el código de la funcionalidad o lógica de negocio. La idea principal detrás de TDD es seguir un ciclo iterativo y repetitivo de tres pasos

Escribir una prueba: Comenzamos escribiendo una prueba que describa la funcionalidad que deseamos implementar. Esta prueba fallará inicialmente porque aún no hemos implementado la funcionalidad… sí, es verdad, ¡parece un sinsentido!

Implementar la funcionalidad: Luego, escribimos el código mínimo necesario para que la prueba pase. No nos preocupamos por la calidad o la optimización en este momento, simplemente nos enfocamos en que la prueba pase. En este punto, algo comienza a tener sentido, ya que desarrollamos sólo la lógica necesaria para que el test aparezca en verde.

Refactorizar el código: Una vez que la prueba ha pasado, revisamos y mejoramos nuestro código manteniendo las pruebas existentes. El objetivo es mantener un código limpio, eficiente y libre de errores. Y en este punto ¡es cuando todo comienza a tener sentido! Ya que podemos mantener la lógica de negocio fundamental sin errores, refactorizando nuestro código para que se ajuste a las mejores prácticas… ¡pero sin romperlo! ya que las pruebas nos evitan desviarnos del camino correcto de la funcionalidad ya implementada.



Beneficios del TDD:

El TDD proporciona una serie de beneficios que mejoran la calidad del código y la eficiencia del desarrollo:

Confianza en el código: Al escribir pruebas antes de implementar la funcionalidad, nos aseguramos de que nuestro código funcione como se espera, lo que aumenta la confianza en su calidad.

Detección temprana de errores: Al tener pruebas que se ejecutan automáticamente, podemos detectar errores o fallos en el código rápidamente, lo que facilita su corrección antes de que se conviertan en problemas mayores.

Mantenimiento sencillo: Las pruebas actúan como una documentación viva de la funcionalidad de nuestro código. Si en el futuro realizamos cambios, las pruebas nos ayudarán a verificar que todo sigue funcionando correctamente.

Diseño centrado en la funcionalidad: El enfoque de escribir pruebas antes de implementar nos obliga a pensar en la funcionalidad deseada antes de escribir código, lo que puede conducir a un mejor diseño de nuestro software.

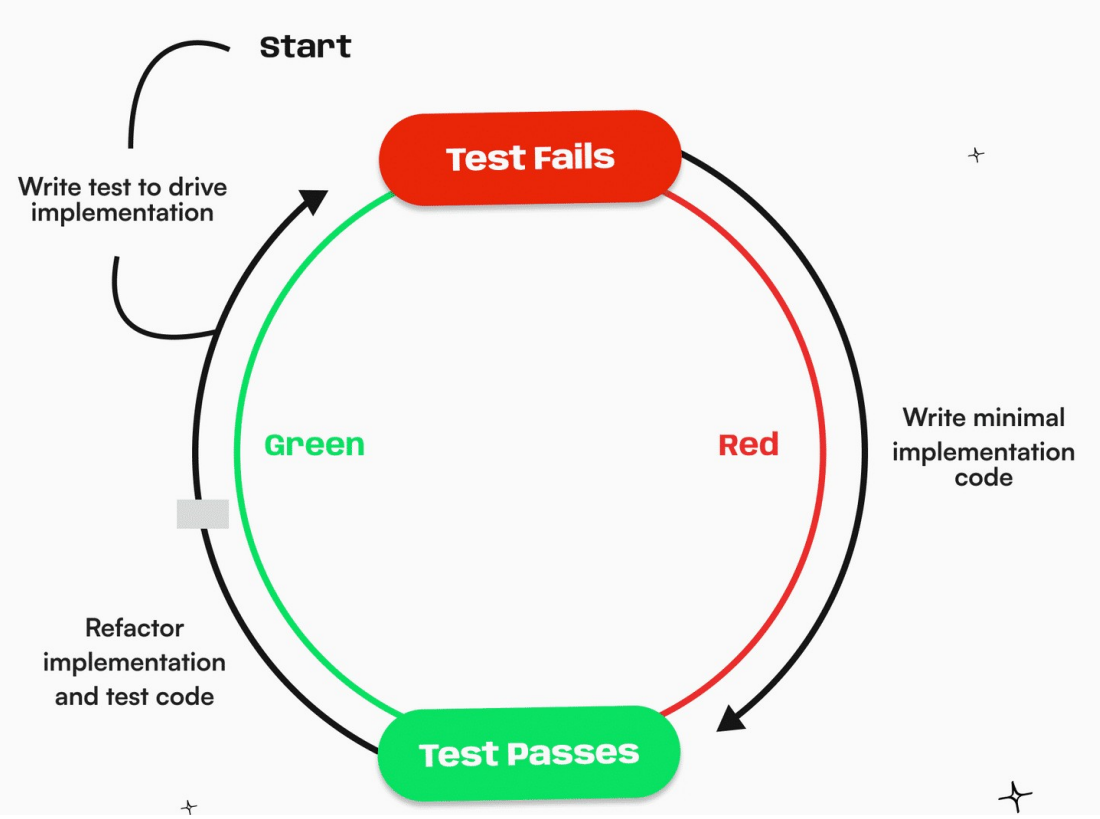
Red, Green, Refactor... otra vez! Red, Green, Refactor!

Ahora que conocimos las bases del TDD aprendamos sobre el Enfoque Red-Green-Refactor, el corazón de esta práctica

El enfoque Red-Green-Refactor es una parte fundamental del Desarrollo Dirigido por Pruebas (TDD). Este enfoque sigue el ciclo iterativo de tres pasos que mencionamos anteriormente:

Red: En esta fase, escribimos una prueba (test) que describa una funcionalidad específica que deseamos implementar. Al principio, la prueba fallará (en rojo), ya que aún no hemos escrito la funcionalidad.

Green: A continuación, escribimos el código mínimo necesario para que la prueba pase (es decir, la prueba se vuelva verde). No nos preocupamos por la calidad del código en este punto, solo nos centramos en hacer que la prueba sea exitosa.
Refactor: Una vez que la prueba pasa, revisamos y mejoramos nuestro código. Realizamos refactorizaciones para asegurarnos de que el código sea limpio, eficiente y siga las buenas prácticas de programación.



Identificación de unidades de código a probar

En TDD, identificar las unidades de código a probar es una etapa crucial. Una "unidad" se refiere a la parte más pequeña y aislada del código que puede ser probada de manera independiente. En Java, una unidad puede ser una clase, un método o incluso una función.
La elección de las unidades a probar depende del diseño y la arquitectura de nuestro código. Idealmente, las unidades de código deben ser lo más pequeñas y cohesivas posible para que las pruebas puedan ser precisas y efectivas.

Escritura de pruebas antes de la implementación

En TDD, escribir pruebas antes de implementar la funcionalidad es una práctica fundamental. Esta estrategia garantiza que la funcionalidad sea desarrollada según los requisitos y que no se añadan características innecesarias o no deseadas.

La escritura de pruebas primero también nos ayuda a tener una visión clara de cómo debe comportarse el código antes de escribirlo. Esto impulsa un enfoque centrado en los resultados y nos ayuda a evitar problemas futuros.

Implementación mínima para pasar las pruebas

Cuando escribimos la implementación para pasar las pruebas, debemos asegurarnos de implementar solo lo necesario para que la prueba pase. No es necesario implementar toda la funcionalidad en esta etapa inicial.

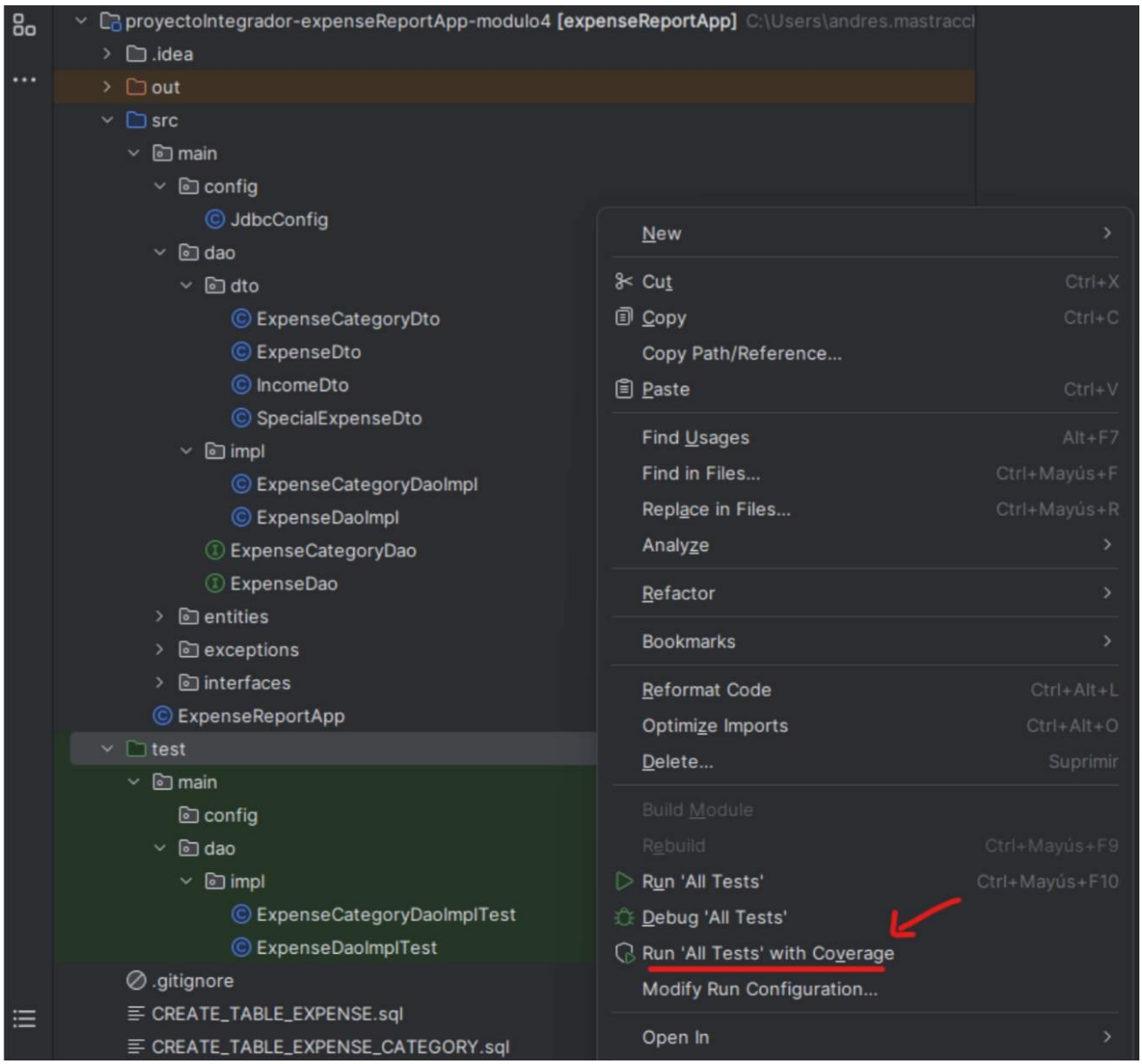
Este enfoque de implementación mínima nos permite iterar rápidamente y centrarnos en escribir solo el código necesario. Posteriormente, en la fase de refactorización, podremos mejorar el código y agregar la lógica restante.
Con el enfoque Red-Green-Refactor, la identificación adecuada de unidades de código, la escritura de pruebas antes de la implementación y la implementación mínima para pasar las pruebas, podemos asegurar que nuestras pruebas sean efectivas y que nuestro código sea robusto, de alta calidad y fácil de mantener

¿Cómo sé si mis test son útiles? ¿Si faltan algunos? ¿Qué parte del proyecto está siendo testeado?

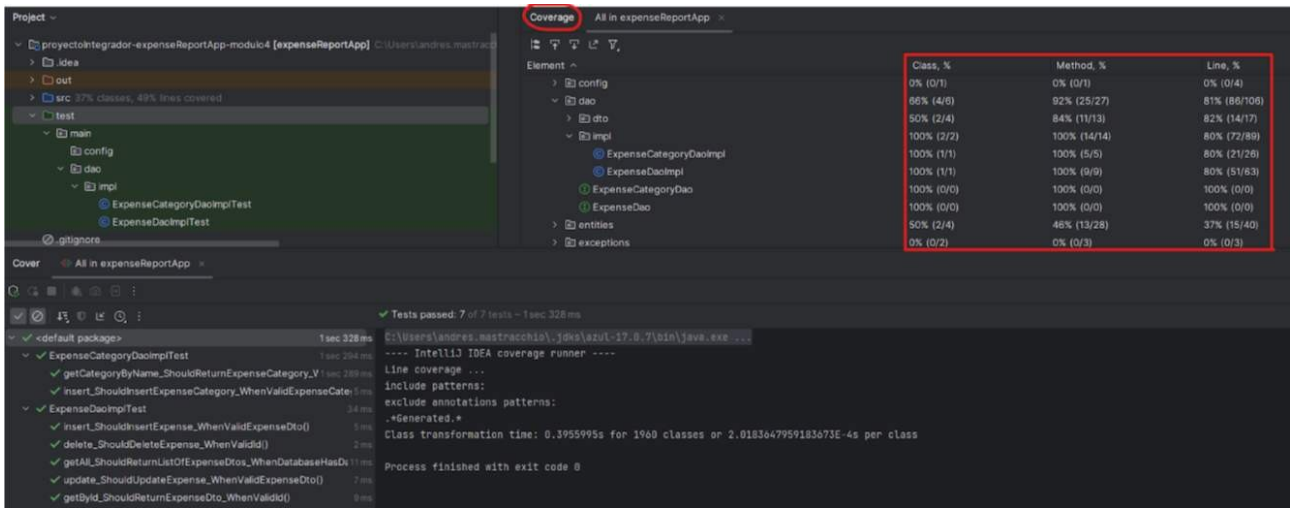
La cobertura de código es una métrica que nos indica la cantidad de código que es ejecutada por nuestras pruebas. Es una medida importante para evaluar la calidad y exhaustividad de nuestras pruebas.

Existen herramientas, como JaCoCo o SonarQube entre varias otras, que nos proporcionan informes detallados que muestran qué líneas de código fueron ejecutadas por nuestras pruebas y cuáles no. De esta manera, podemos identificar áreas del código que no están siendo probadas y asegurarnos de que nuestras pruebas abarquen la mayor cantidad de código posible.
Una alta cobertura de código no garantiza que nuestras pruebas sean perfectas, pero puede indicar que hemos probado muchas de las rutas posibles en el código. Por otro lado, una baja cobertura de código puede sugerir que hay partes del código que no están siendo probadas y que pueden contener errores no detectados.
Es esencial analizar los resultados de cobertura con un enfoque crítico y considerar cómo mejorar nuestras pruebas para abarcar más áreas del código.

En nuestro caso, no estaremos trabajando con herramientas tan complejas por el momento, sino que nos ayudaremos de las herramientas básicas de las cuales nos provee IntelliJ IDEA. Para poder hacer una verificación de cobertura de nuestro código, podemos hacer lo siguiente:



Hacemos click derecho en la carpeta raíz de tests y seleccionamos la opción “Run ‘All Tests’ with Coverage”.



Esta herramienta, hará correr todos los tests, comprobándolos y generando un pequeño reporte con un porcentaje de cobertura en tres categorías: cantidad de

clases testeadas, cantidad de métodos testeados y cantidad de líneas testeadas. Tres aproximaciones de criterios por los cuales podemos guiarnos para definir un estándar aceptable para nuestra aplicación.
Además de la cobertura de código, existen otras métricas importantes que nos ayudan a evaluar la calidad de nuestras pruebas:
Tiempo de ejecución: El tiempo que lleva ejecutar todas las pruebas. Un tiempo de ejecución largo puede ser indicativo de pruebas ineficientes o redundantes.
Resultados de pruebas: El número de pruebas exitosas, fallidas o con errores. Un alto porcentaje de pruebas exitosas es un indicador positivo.
Complejidad ciclomática: La complejidad del código, que puede indicar áreas propensas a errores y que necesitan más pruebas.
Proporción de pruebas unitarias y de integración: Es importante tener un equilibrio entre pruebas unitarias (prueban componentes individuales) y pruebas de integración (prueban la interacción entre componentes).
Al evaluar estas métricas, podemos mejorar nuestras pruebas y garantizar una mayor calidad en nuestro código.

Mejores prácticas y consejos

Algunos consejos generales de la comunidad para mejorar nuestros tests.

Nombres de pruebas descriptivos:

Los nombres de las pruebas deben ser descriptivos y claros para que cualquier persona que lea el código pueda entender qué se está probando sin tener que revisar el código de la prueba en detalle. Un nombre descriptivo debe indicar claramente el escenario que se está probando y el resultado esperado.

Por ejemplo, si estamos probando una función que suma dos números, un buen nombre de prueba podría ser "testSumaNumerosPositivos" o "testSumaNumerosNegativos".

Mantenimiento de pruebas a lo largo del tiempo:

A medida que nuestro código evoluciona con el tiempo, también deben hacerlo nuestras pruebas. Es importante mantener las pruebas actualizadas y reflejar cualquier cambio realizado en el código.

Cuando hagamos modificaciones en el código, debemos revisar y, si es necesario, actualizar las pruebas para asegurarnos de que sigan siendo relevantes y precisas. Además, debemos eliminar pruebas obsoletas o que ya no son relevantes

Técnicas de diseño para facilitar las pruebas:

Al diseñar nuestro código, podemos aplicar técnicas que faciliten la escritura y mantenimiento de las pruebas:

Inyección de dependencias: Diseñar nuestro código para que dependa de interfaces y utilizar la inyección de dependencias nos permite simular fácilmente objetos en nuestras pruebas y aislar el código bajo prueba.

Desarrollo por capas: Organizar nuestro código en capas facilita la escritura de pruebas unitarias, ya que podemos probar cada capa de manera aislada.

Diseño modular: Desarrollar funciones o métodos pequeños y cohesivos facilita la escritura de pruebas que cubran todos los casos posibles.

Evitar acoplamiento excesivo: Reducir el acoplamiento entre componentes del código nos permite cambiar partes del código sin afectar otras áreas, lo que facilita las pruebas.

Uso de patrones de diseño: Al aplicar patrones de diseño en nuestro código, podemos mejorar la legibilidad y testabilidad, lo que facilita la escritura de pruebas.

En resumen, al seguir buenas prácticas como usar nombres descriptivos para las pruebas, mantener las pruebas actualizadas con el código y aplicar técnicas de diseño adecuadas, podemos desarrollar pruebas más efectivas y de alta calidad.

Los Orígenes de Spring Framework: Un Cambio Paradigmático en Java

Finales de los años noventa

La historia de Spring Framework es un testimonio de cómo una idea audaz y una visión clara pueden transformar la manera en que se desarrollan aplicaciones Java. A finales de la década de 1990, cuando la comunidad de desarrollo de Java estaba buscando soluciones para los desafíos cada vez mayores en la creación de aplicaciones empresariales, un desarrollador llamado Rod Johnson comenzó a esbozar las bases de lo que se convertiría en una revolución en el mundo del desarrollo.

En 2002, el fruto de su trabajo se materializó en forma de Spring Framework. Este framework de desarrollo de aplicaciones Java de código abierto no solo introdujo un enfoque completamente nuevo para el desarrollo, sino que también cambió la forma en que los desarrolladores pensaban sobre el diseño de software.

El Nacimiento de Conceptos Fundamentales: Inversión de Control y Aspect-Oriented Programming

Uno de los conceptos clave que definió Spring desde sus inicios fue la "Inversión de Control" (IoC). Rod Johnson y su equipo introdujeron la idea de que la creación y la administración de objetos deberían ser responsabilidad del contenedor, no del programador. Esta idea revolucionaria permitió una mayor modularidad, reutilización y mantenibilidad en el desarrollo de aplicaciones.

Otro concepto innovador que Spring introdujo fue la Programación Orientada a Aspectos (AOP). Rod Johnson reconoció que muchas preocupaciones, como la seguridad y el registro de logs, eran preocupaciones transversales que se entrecruzan con la lógica principal del negocio. Con AOP, Spring permitió a los desarrolladores separar estas preocupaciones transversales y aplicarlas de manera más eficiente y modular

El Crecimiento y la Madurez de Spring Framework

A medida que Spring Framework ganaba seguidores en la comunidad de desarrollo, comenzaron a surgir nuevas funcionalidades y módulos. La comunidad contribuyó activamente, y los desarrolladores de Spring respondieron con soluciones innovadoras para desafíos emergentes. La introducción de proyectos como Spring Boot, Spring Data, Spring Security y Spring Cloud expandió aún más las capacidades del framework, abordando áreas clave como la simplicidad de configuración, el acceso a datos y la seguridad.

Spring Framework no solo cambió la forma en que se desarrollaban aplicaciones, sino que también influyó en la evolución de Java en sí. Muchas de las

características que Spring introdujo en sus primeras versiones se incorporaron posteriormente en Java EE y en el propio lenguaje Java.

Un Legado de Innovación y Empoderamiento

Hoy en día, **Spring Framework** es un pilar en la comunidad de desarrollo de Java. Su enfoque en la modularidad, la inversión de control y la programación orientada a aspectos ha influido en cómo se diseñan y construyen aplicaciones en una variedad de contextos. La historia de Spring es una prueba viviente de cómo una visión audaz y una mentalidad innovadora pueden dar forma a la forma en que se crea software.

A medida que Spring continúa evolucionando y adaptándose a las nuevas demandas del desarrollo de software, su historia sigue siendo una fuente de inspiración para los desarrolladores de todo el mundo. La revolución que comenzó con la visión de Rod Johnson ha dejado una marca indeleble en la industria y sigue siendo una fuerza impulsora en la innovación y la excelencia en el desarrollo de aplicaciones Java.

Conceptos generales

Spring Framework, también llamado normalmente sólo Spring, es un marco de trabajo (framework) de código abierto para el desarrollo de aplicaciones Java empresariales. Fue creado para abordar problemas comunes en el desarrollo de software empresarial, como la complejidad, la falta de modularidad y la dificultad en las pruebas unitarias. Spring se ha convertido en uno de los marcos de trabajo más populares y ampliamente utilizados en el ecosistema de Java.
Spring Framework se centra en varios aspectos clave del desarrollo de aplicaciones empresariales, incluyendo la inversión de control (IoC), la inyección de dependencias, la programación orientada a aspectos (AOP) y una amplia gama de módulos y bibliotecas para simplificar el desarrollo.
Se basa en una serie de principios de diseño que promueven una arquitectura limpia y modular, lo que facilita la creación de aplicaciones mantenibles y escalables. Algunos de los principios clave son:

Inversión de control

La Inversión de Control (IoC) es un principio fundamental en el diseño de aplicaciones que ha sido ampliamente adoptado por muchos frameworks modernos, incluido Spring. IoC es una técnica que cambia la responsabilidad del control de la creación y gestión de objetos de la aplicación del programador a un contenedor gestionado por el mismo framework.

Se basa en la premisa de que en lugar de que un componente sea responsable de crear y gestionar sus dependencias, estas dependencias son proporcionadas externamente. Esto significa que el control sobre cómo se crean y manejan los objetos se invierte o externaliza a un contenedor IoC, como el contenedor de Spring

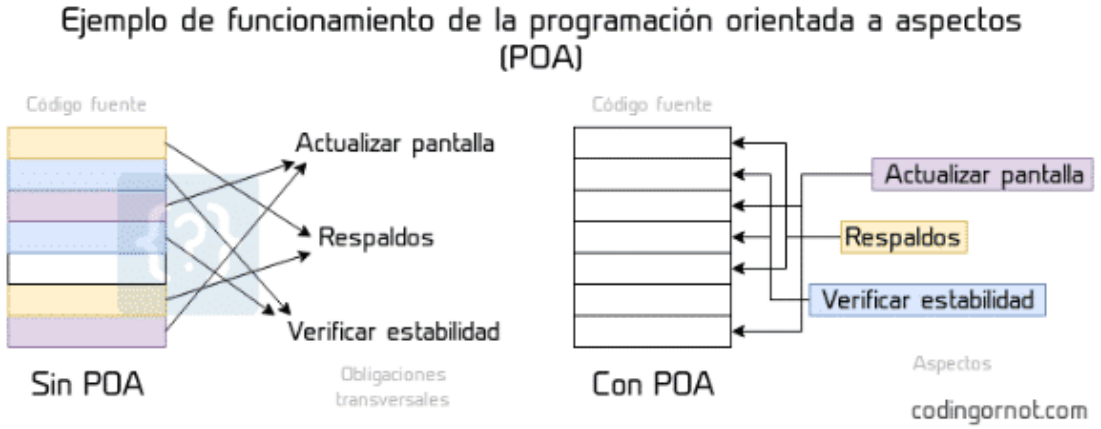
Programación Orientada a Aspectos (AOP) en Spring:

¿A que nos referimos con AOP?

La Programación Orientada a Aspectos (AOP) es una técnica de programación que aborda la modularidad de una manera diferente a la Programación Orientada a Objetos (OOP). AOP se centra en la separación de preocupaciones que no se ajustan perfectamente a los objetos individuales, permitiendo un desarrollo más limpio y organizado. En el contexto de Spring, AOP es una herramienta poderosa para abordar la lógica transversal que afecta a múltiples componentes en una aplicación.

¿Qué es la Programación Orientada a Aspectos (AOP)?

La AOP aborda las preocupaciones que no están directamente relacionadas con la lógica de un componente, como el registro, la seguridad, la transacción, entre otras. En lugar de mezclar estas preocupaciones con el código de los componentes, AOP permite separarlas en "aspectos" que se aplican de manera transversal a múltiples partes del código.



Los beneficios de AOP en Spring

1. **Separación de Preocupaciones:** AOP ayuda a mantener el código limpio y organizado al separar las preocupaciones no funcionales de la lógica principal. Esto mejora la claridad del código y facilita su mantenimiento.
2. **Reutilización:** Los aspectos se pueden aplicar en varios lugares de la aplicación, lo que permite la reutilización de lógica transversal en diferentes contextos.
3. **Mejora la Mantenibilidad:** Al aislar las preocupaciones transversales en aspectos, los cambios en estas áreas no afectan la lógica de los componentes individuales.
4. **Aspectos Dinámicos:** En Spring, los aspectos pueden ser aplicados dinámicamente en tiempo de ejecución, lo que ofrece flexibilidad en la gestión de las preocupaciones transversales

Los proyectos de Spring

Spring cuenta con un ecosistema robusto que incluye diversos proyectos complementarios que abordan diferentes aspectos y soluciones del desarrollo de aplicaciones. Algunos de los proyectos más destacados son:

Spring Boot

En las aplicaciones Spring tradicionales, la configuración puede ser un proceso laborioso. Los desarrolladores a menudo necesitan definir manualmente una gran cantidad de configuraciones y propiedades para que la aplicación funcione correctamente. Esto puede ser tedioso y propenso a errores.

Spring Boot es uno de los proyectos más destacados en el ecosistema. Esta herramienta innovadora simplifica drásticamente la creación y configuración de aplicaciones Spring. Al automatizar gran parte de la configuración, Spring Boot permite a los desarrolladores concentrarse en la lógica de negocio en lugar de preocuparse por detalles técnicos. Proporciona integración con servidores embebidos, configuración basada en propiedades y una amplia variedad de starters preconfigurados para diferentes tipos de aplicaciones. Esto agiliza la puesta en marcha y acelera el desarrollo de aplicaciones listas para producción

Spring Data: Simplificando el Acceso a Datos

En el mundo del desarrollo de aplicaciones, el acceso y manejo de datos es una parte fundamental. Spring Data es un proyecto dentro del ecosistema Spring que aborda este desafío al proporcionar una forma unificada y simplificada de interactuar con diferentes tipos de bases de datos. En combinación con Spring Boot, Spring Data se convierte en una herramienta esencial para el acceso a datos eficiente y escalable.

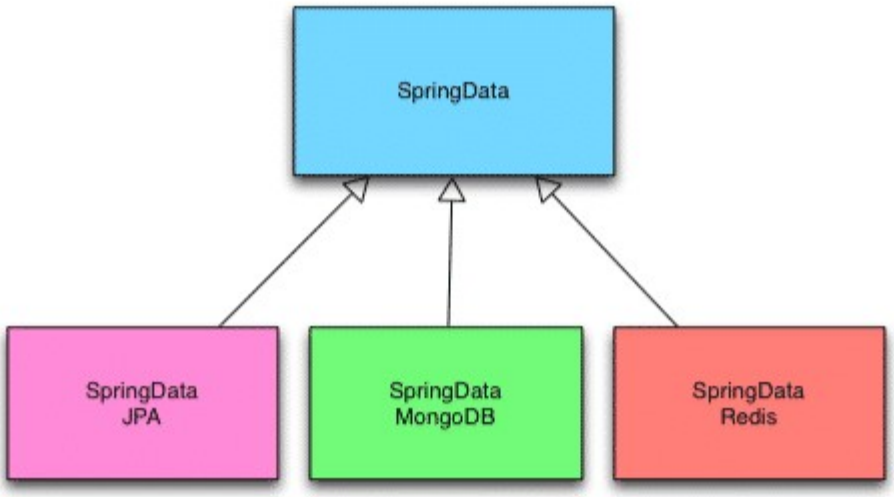
¿Qué es Spring Data?

Spring Data es un conjunto de proyectos que facilita el acceso y la manipulación de datos en aplicaciones Spring. Proporciona una capa de abstracción que permite trabajar con bases de datos relacionales y NoSQL utilizando un enfoque coherente y consistente.

Características Clave de Spring Data

Abstracción de Datos: Spring Data ofrece una abstracción para diferentes tipos de bases de datos, lo que permite a los desarrolladores interactuar con bases de datos relacionales (como MySQL, PostgreSQL) y NoSQL (como MongoDB, Redis) con la misma API.

- **Menos Código Boilerplate:** Spring Data reduce la cantidad de código repetitivo y boilerplate necesario para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en la base de datos. Esto permite un desarrollo más rápido y eficiente.
- **Consultas Dinámicas:** Spring Data ofrece la posibilidad de crear consultas dinámicas mediante la nomenclatura de métodos. Esto significa que puedes definir consultas complejas utilizando nombres de métodos en lugar de escribir consultas SQL manualmente.
- **Integración con Spring Boot:** Spring Data se integra perfectamente con Spring Boot, lo que significa que gran parte de la configuración y la inicialización se gestionan automáticamente. Esto acelera el desarrollo y reduce la posibilidad de errores de configuración.
- **Soporte para Transacciones**: Spring Data proporciona soporte para transacciones, lo que permite gestionar transacciones de manera coherente al realizar operaciones en la base de datos.



Spring Security:

Garantizando la Seguridad en Aplicaciones Spring

En el desarrollo de aplicaciones modernas, la seguridad es una preocupación fundamental. Spring Security es un proyecto integral del ecosistema Spring que se dedica a proporcionar soluciones robustas y flexibles para la autenticación y la autorización en aplicaciones Spring y Spring Boot. Con Spring Security, los desarrolladores pueden implementar capas de seguridad en sus aplicaciones de manera efectiva y sin tener que reinventar la rueda.

¿Qué es Spring Security?

Spring Security es un marco de trabajo que se encarga de gestionar la seguridad en aplicaciones basadas en Spring. Ofrece una amplia gama de características, desde autenticación básica hasta configuraciones avanzadas de autorización y protección contra ataques comunes.

Características Clave de Spring Security

- **Autenticación y Autorización:** Spring Security proporciona un sólido mecanismo de autenticación, permitiendo a los usuarios autenticarse de diversas maneras, como mediante formularios, tokens o autenticación basada en API. Además, brinda opciones flexibles para definir políticas de autorización basadas en roles y permisos.
- **Configuración Declarativa:** Spring Security permite la configuración declarativa mediante anotaciones o archivos de configuración XML. Esto facilita la personalización de la seguridad sin necesidad de modificar el código fuente.
- **Protección contra Ataques Comunes**: Spring Security incluye protecciones integradas contra ataques comunes como Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF) y otros.
- **Integración con Spring Boot**: Spring Security se integra perfectamente con Spring Boot. Al agregar la dependencia de Spring Security en un proyecto Spring Boot, muchas configuraciones se realizan automáticamente, lo que facilita la implementación de seguridad en la aplicación.

- **Soporte para OAuth**: Spring Security proporciona soporte para OAuth, un protocolo de autorización ampliamente utilizado para autenticación segura en aplicaciones web y móviles.

Spring Cloud: Construyendo Sistemas Distribuidos y Robustos

En la era actual de la informática, los sistemas distribuidos se han vuelto omnipresentes. Spring Cloud es un conjunto de herramientas dentro del ecosistema Spring que aborda los desafíos de la construcción de aplicaciones distribuidas y microservicios. Al aprovechar las capacidades de Spring Boot, Spring Cloud proporciona soluciones para la gestión de la configuración, el descubrimiento de servicios, la tolerancia a fallos y otros aspectos esenciales de la arquitectura de microservicios.

¿Qué es Spring Cloud?

Spring Cloud es una colección de proyectos que ayudan a los desarrolladores a construir sistemas distribuidos y escalables utilizando Spring Boot como base. Proporciona herramientas para abordar problemas comunes en la arquitectura de microservicios, como la gestión de la configuración, el balanceo de carga, la tolerancia a fallos y más.

Características Clave de Spring Cloud

Configuración Centralizada: el servicio de Config permite gestionar la configuración de múltiples microservicios en un lugar centralizado, lo que facilita la actualización y el mantenimiento.

Descubrimiento y Balanceo de Carga: Netflix Eureka es una solución de Spring cloud que ofrece capacidades de descubrimiento de servicios, lo que permite que los microservicios se registren automáticamente y se comuniquen entre sí. Además, Netflix Ribbon, otra solución dentro del ecosistema, proporciona un balanceo de carga inteligente entre instancias de servicios.

Tolerancia a Fallos: Netflix Hystrix es un servicio que permite implementar tolerancia a fallos y resiliencia en sistemas distribuidos, evitando que un fallo en un microservicio afecte a toda la aplicación.

Gateways y Enrutamiento: el servicio de Gateway ofrece una solución para crear gateways de API y administrar el enrutamiento y filtrado de solicitudes entrantes.

Tracer y Monitoreo: Sleuth proporciona herramientas para rastrear las solicitudes a través de diferentes microservicios, lo que facilita el monitoreo y la depuración.

Introducción a Spring Framework y Spring Boot

Antes de adentrarnos en las maravillas de Spring Boot, es esencial que entendamos el trasfondo de Spring Framework y cómo ha evolucionado con el tiempo. Imagina que estamos retrocediendo en el tiempo antes de la llegada de Spring Boot. El enfoque predominante en la configuración de aplicaciones Spring implicaba la creación manual de beans y sus dependencias mediante archivos XML. Aunque esto proporcionaba flexibilidad, también podía ser complejo y propenso a errores debido a la cantidad significativa de código XML involucrado.

Explorando Spring Framework con Configuración basada en XML

En esta parte de la clase, exploraremos la arquitectura y los módulos fundamentales de Spring Framework. Aprenderemos cómo configurar una aplicación mediante archivos XML, definiendo beans y sus dependencias. Aunque este enfoque histórico era valioso en su época, su complejidad a menudo resultaba abrumadora y requería un cuidadoso seguimiento.

Desafiando el Status Quo con Spring Boot

Aquí es donde entra en escena Spring Boot, un héroe moderno en el mundo de las aplicaciones Spring. Spring Boot ha revolucionado la forma en que desarrollamos aplicaciones, simplificando drásticamente la configuración y reduciendo el código boilerplate. En lugar de confiar en archivos XML, Spring Boot utiliza anotaciones Java para definir configuraciones, lo que significa que gran parte de la configuración se realiza directamente en el código.

¡Prepárate para sumergirte en un viaje emocionante a medida que exploramos los conceptos clave de Spring Framework y Spring Boot, y descubrimos cómo esta última tecnología ha revolucionado la eficiencia y la simplicidad en el desarrollo de aplicaciones!

Arquitectura de Spring

Como ya vimos, Spring Framework se basa en principios clave como la Inversión de Control (IoC) y la Inyección de Dependencias (DI), elementos fundamentales que estaremos abordando más en profundidad en otra lección. Pero es muy importante conocer un poco la arquitectura del framework. Esta se divide en módulos, cada uno centrado en un aspecto específico del desarrollo:

Spring Core Container
Este módulo incluye el contenedor Bean y el mecanismo de Inyección de Dependencias. El contenedor Bean administra la creación, configuración y gestión de objetos (beans), mientras que la Inyección de Dependencias permite que los objetos reciban sus dependencias en lugar de crearlas.
Spring Data Access/Integration
Además de los contenedores, el acceso a los datos es muy importante.
Este módulo ofrece soporte para la interacción con bases de datos y sistemas de backend. Proporciona capas de abstracción para acceder a diferentes fuentes de datos, como JDBC, ORM (Object-Relational Mapping) y JMS (Java Message Service).

Spring AOP (Aspect-Oriented Programming)
Tenemos a nuestra programación orientada a aspectos.
Spring AOP permite separar preocupaciones transversales, como la seguridad y el registro, del código empresarial principal. Esto se logra mediante la definición de aspectos que encapsulan el comportamiento a ser aplicado en varios puntos de corte (join points) en la aplicación.

Spring Web
¿Y si queremos hacer una aplicación web?
El módulo Spring Web facilita el desarrollo de aplicaciones web. Incluye Spring MVC para la creación de aplicaciones basadas en el patrón Modelo-Vista-Controlador (MVC), y Spring WebMVC para la construcción de aplicaciones web RESTful.

Configuración basada en XML en Spring Framework

Antes de la llegada de Spring Boot, la configuración en Spring Framework se realizaba principalmente mediante archivos XML.
Cabe mencionar que XML es una herramienta versátil para estructurar datos de manera legible y jerárquica, y su flexibilidad lo hace útil en una amplia variedad de aplicaciones, desde el intercambio de datos hasta la personalización de aplicaciones y la creación de contenido en línea.

XML, significa "Extensible Markup Language" (Lenguaje de Marcado Extensible), y es un lenguaje de marcado utilizado para almacenar y transportar datos de manera estructurada. A diferencia de los lenguajes de programación, XML no realiza operaciones lógicas o cálculos, sino que se centra en describir la estructura y el contenido de los datos de manera legible tanto para humanos como para máquinas.

Un archivo XML está compuesto por etiquetas que encierran datos dentro de elementos. Estas etiquetas deben ser auto-definidas, lo que significa que los usuarios pueden crear sus propias etiquetas personalizadas para adaptarse a sus necesidades específicas.

Los usos más comunes de XML incluyen:

Intercambio de datos: se utiliza para intercambiar datos entre diferentes sistemas, aplicaciones y plataformas. Gracias a su estructura jerárquica y legible, es ideal para representar información compleja de manera uniforme.

•**Configuración y personalización:** muchas aplicaciones utilizan archivos XML para almacenar configuraciones y preferencias. Por ejemplo, el caso que vemos en este curso con Spring. Esto permite a los usuarios personalizar la aplicación según sus necesidades.

•**Almacenamiento de datos:** se puede usar para almacenar datos en forma de documentos legibles. Esto es útil cuando se necesita mantener un registro estructurado de información, como bases de datos pequeñas o registros de actividad.

•**Documentación**: se emplea para crear documentación técnica, manuales y guías, ya que permite organizar y presentar información de manera ordenada.

•**Web services**: es esencial en la comunicación entre aplicaciones web a través de servicios web, ya que proporciona un formato estándar para intercambiar datos entre diferentes sistemas. Es una alternativa, un poco cada vez más deprecada, al popular JSON.

•**Formato de archivos**: algunos tipos de archivos, como documentos de Microsoft Office (.docx, .xlsx), utilizan XML para almacenar información, lo que permite un fácil análisis y procesamiento.

Entonces, regresando a Spring, con estos archivos de tipo XML, los desarrolladores definían manualmente beans, dependencias y configuraciones de la aplicación utilizando estos archivos. Aunque permitía una separación de responsabilidades, este enfoque tenía ciertas limitaciones.

Incorporación de las dependencias de Spring

Por supuesto, para trabajar con Spring, lo primero que debemos hacer es buscar y descargar las dependencias necesarias para poder trabajar con el Framework. Este trabajo requiere investigación y tiempo para leer la documentación, así poder entonces saber cuales jars debemos importar a nuestro classpath. Esto es importante tener encuenta, ya que la curaduría de dependencias es un tiempo de desarrollo que se debe tomar en cuenta.

Para nuestro caso, ahorrémonos este tiempo para poder concentrarnos en los conceptos de Spring. Las dependencias necesarias pueden ser descargadas desde este enlace y deben ser incorporadas a la aplicación, de la misma manera que incorporamos H2 o Junit.

Definición y Creación de Beans Mediante XML

Por supuesto, para trabajar con Spring, lo primero que debemos hacer es buscar y descargar las dependencias necesarias para poder trabajar con el Framework. Este trabajo requiere investigación y tiempo para leer la documentación, así poder entonces saber cuales jars debemos importar a nuestro classpath. Esto es importante tener encuenta, ya que la curaduría de dependencias es un tiempo de desarrollo que se debe tomar en cuenta.
Para nuestro caso, ahorrémonos este tiempo para poder concentrarnos en los conceptos de S´ring. Las dependencias necesarias pueden ser descargadas desde este enlace y deben ser incorporadas a la aplicación, de la misma manera que incorporamos H2 o Junit.

```
<bean id="miBean" class="com.example.MiBean">
   <!-- Propiedades y configuraciones adicionales aquí -->
</bean>
<libros>
    <libro>
        <titulo>El Gran Gatsby</titulo>
        <autor>F. Scott Fitzgerald</autor>
        <anio>1925</anio>
    </libro>
    <libro>
        <titulo>Cien años de soledad</titulo>
        <autor>Gabriel García Márquez</autor>
        <anio>1967</anio>
    </libro>
    <libro>
        <titulo>1984</titulo>
        <autor>George Orwell</autor>
        <anio>1949</anio>
    </libro>
</libros>
```

Este XML representa una colección de libros, donde cada **<libro>** tiene elementos **<titulo>**, **<autor>** y **<anio>** que contienen información específica sobre el libro. Los elementos **<libro>** están contenidos dentro de un elemento **<libros>** que actúa como el elemento raíz del documento.

Inyección de Dependencias Mediante XML

¿Y cómo se hacía la inyección de dependencias con xml?

La Inyección de Dependencias se lograba configurando las propiedades de los beans en los archivos XML. Esto permitía inyectar otras beans o valores primitivos en el bean actual.

```
<bean id="servicioA" class="com.example.ServicioA">
   <property name="otroBean" ref="miBean"/>
</bean>
```

Configuración Automática y Opinionated Defaults en Spring Boot

Con la llegada de Spring Boot, la configuración de aplicaciones Spring se simplificó significativamente, lo que permitió a los desarrolladores centrarse más en la lógica de negocio.

Creación de un Proyecto de Spring Boot

Spring Boot facilita la creación de proyectos utilizando Spring Initializr. Esta herramienta en línea permite generar un proyecto base con las dependencias y configuraciones necesarias. Simplemente selecciona las opciones relevantes, descarga el archivo ZIP y estarás listo para empezar.

Anotaciones para la Configuración sin XML en Spring Boot

Spring Boot promueve el uso de anotaciones en lugar de archivos XML para configurar las aplicaciones. Algunas anotaciones clave incluyen:

- **`@SpringBootApplication`**: Anotación que se coloca en la clase principal de la aplicación y combina otras anotaciones para configurar la aplicación.

- **`@Configuration`:** Anotación que indica que una clase contiene definiciones de beans.

- **`@ComponentScan`:** Anotación que especifica los paquetes que Spring debe escanear en busca de componentes y beans.

Inyección de Dependencias Mediante Anotaciones

La inyección de dependencias es un concepto fundamental en Spring que permite a los objetos colaborar entre sí de manera flexible y desacoplada. Spring Boot simplifica en gran medida la inyección de dependencias mediante el uso de

anotaciones en lugar de requerir configuraciones XML detalladas. Esto facilita la gestión de dependencias y mejora la legibilidad del código. A continuación, explicaremos cómo realizar la inyección de dependencias mediante anotaciones en Spring Boot.

Anotaciones Clave para Inyección de Dependencias

`@Autowired`

La anotación `@Autowired` se utiliza para inyectar automáticamente una dependencia en un componente de Spring. Puede aplicarse a campos, constructores y métodos setters en una clase.

Inyección en Campos

@Component
```
public class MiClase {
    @Autowired
    private OtroBean otroBean;
}
```

video de autowired: https://youtu.be/sLY9umEahso

Inyección en Constructores

@Service
```
public class ServicioA {
    private OtroBean otroBean;

    @Autowired
    public ServicioA(OtroBean otroBean) {
        this.otroBean = otroBean;
    }
}
```

`@Qualifier`

Cuando hay varias implementaciones de una interfaz y se necesita inyectar una instancia específica, se puede utilizar la anotación `@Qualifier` junto con `@Autowired`.

@Service

public class ServicioB {
```
    private MiInterfaz miInterfaz;

    @Autowired
    public ServicioB(@Qualifier("implementacionA") MiInterfaz miInterfaz) {
        this.miInterfaz = miInterfaz;
    }
}
```

video con qualifier y service: https://youtu.be/NmD9LL1Lp5g

Ventajas de la Inyección de Dependencias Mediante Anotaciones

- **Simplicidad:** Las anotaciones como `@Autowired` reducen la necesidad de escribir configuraciones manuales de dependencias.
- **Legibilidad:**La inyección de dependencias mediante anotaciones es más clara y directa en comparación con la configuración XML.
- **Reducido Acoplamiento:** La inyección de dependencias reduce el acoplamiento entre componentes, lo que facilita las pruebas unitarias y el mantenimiento.
- **Flexibilidad:** Cambiar las dependencias inyectadas es más simple, ya que solo se requieren modificaciones en el código y no en la configuración.

Ejemplo

Supongamos que tenemos una clase `ClienteService` que depende de una interfaz `ClienteRepository`. Utilizando la inyección de dependencias mediante anotaciones, podemos hacer lo siguiente:

```
@Service
public class ClienteService {
    private final ClienteRepository clienteRepository;

    @Autowired
    public ClienteService(ClienteRepository clienteRepository) {
        this.clienteRepository = clienteRepository;
    }

    // ...
}
```

En este ejemplo, la dependencia `clienteRepository` se inyecta automáticamente en el constructor de `ClienteService`.

Spring Framework vs. Spring Boot

Con la introducción de Spring Boot, se han experimentado cambios significativos en la forma en que desarrollamos aplicaciones Java. Aquí se presentan las ventajas clave de Spring Boot en comparación con el enfoque tradicional de Spring Framework basado en XML:

- **Reducción de Configuración y Código Boilerplate:** Spring Boot elimina la necesidad de configuraciones extensas y repetitivas, lo que acelera el desarrollo y reduce el código boilerplate.
- **Facilidad de Inicio y Desarrollo Rápido:** La configuración automática y las *opinionated defaults* permiten a los desarrolladores comenzar a trabajar rápidamente y centrarse en la lógica de negocio en lugar de configuraciones tediosas.
- **Integración con Servidores Embebidos:** Spring Boot facilita la creación de aplicaciones autocontenidas que incluyen servidores web embebidos. Esto simplifica la implementación y reduce la complejidad de la infraestructura.

- **Gestión de Dependencias Simplificada:** Spring Boot maneja automáticamente las dependencias y sus versiones, evitando conflictos y mejorando la estabilidad y coherencia de la aplicación.
- **Actuadores y Monitoreo:** Spring Boot ofrece actuadores preconfigurados que permiten supervisar y administrar la aplicación en tiempo real. Esto facilita el monitoreo y la detección de problemas.
- **Configuración Externa:** Spring Boot permite la configuración externa mediante archivos, lo que facilita la personalización y el ajuste de la aplicación en diferentes entornos sin modificar el código fuente.

Te invitamos a ver este interesante fragmento de charla sobre Spring: https://youtu.be/SAk683SH3ag

Creando un proyecto de Spring Boot

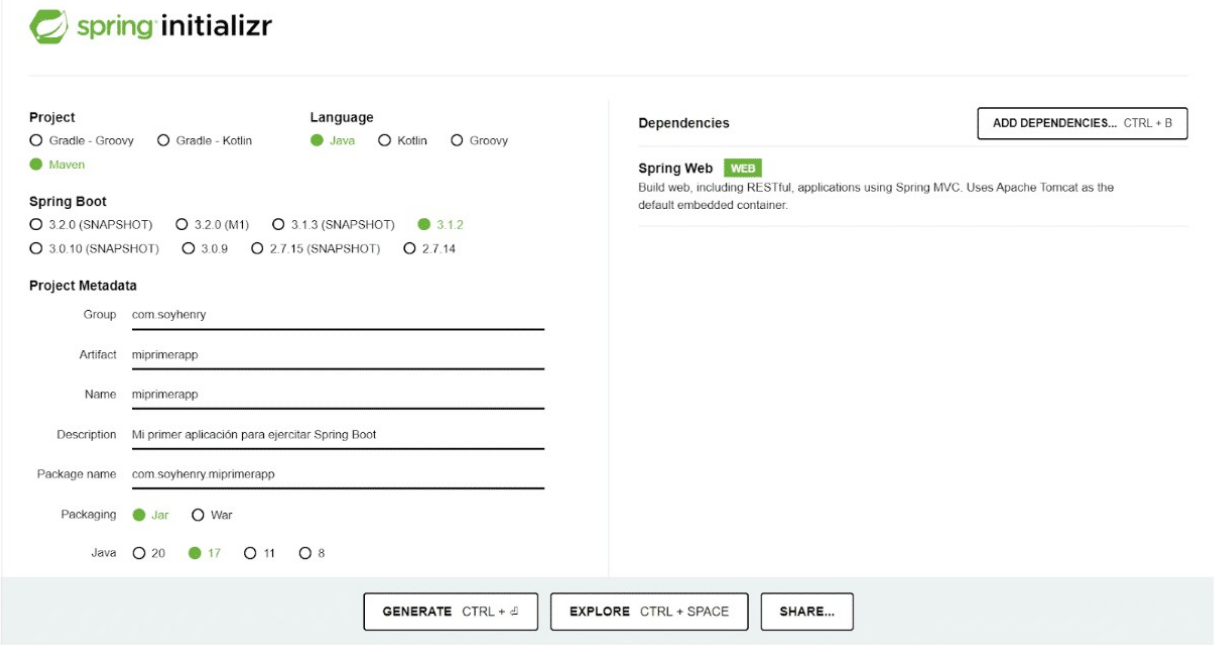
Pasos para Crear un Proyecto de Spring Boot con Spring Initializr

paso 1

Acceder a Spring Initializr: Abre tu navegador web y visita la página de Spring Initializr en https://start.spring.io/.

Paso 2

Configurar el Proyecto:



En la página de Spring Initializr, se presentarán varias opciones de configuración para tu proyecto. Aquí puedes especificar detalles como el lenguaje de programación, la versión de Spring Boot y las dependencias que desees agregar a tu proyecto.

Paso 3

En “Project” elegimos la opción de Maven como gestor de dependencias. En este workshop no estaremos ahondando en este tema, pero es importante entender que

Maven no estará ayudando a incorporar librerías propias de Spring o de terceros a nuestro proyecto.

Paso 4

Para el “Lenguaje”, por supuesto, elegiremos Java, pero podrás ver que existen otros lenguajes disponibles para desarrollar con Spring.

“Versión de Spring Boot”, elegiremos la última versión que NO sea Snapshot o M1.

Paso 5

Los campos de “Group”, “Artifact” y “Name” definen el "GroupId" definen la nomenclatura de la organización y establece el nombre principal del directorio raíz de nuestra app. Podrás ver que a medida que ingresas la nomenclatura, el package name se autocompleta.

También puedes escribir una “Descripción” breve sobre la función u objetivo de tu app.

Paso 6

Packaging: puedes elegir empaquetar tu proyecto como un JAR (archivo Java) o un WAR (archivo web) según los requerimientos para el deployment posterior de tu aplicación. Para más información sobre los usos y diferencias entre estos tipos de archivos, puedes visitar el siguiente link

Paso 7

Para ir terminando, elegimos la versión de Java con la que vamos a trabajar (en este punto, por el momento, es libre elección).

Paso 8

Finalmente, en la sección derecha de la pantalla, podemos elegir las “dependencias” o librerias de spring para el desarrollo de nuestra app. Como podrás ver, existen multitud de librerias desarrolladas, pero por el momento solo nos contentaremos con elegir la dependencia de Spring Web, que nos ayudará en el desarrollo de una REST API.

Paso 9

Generar el Proyecto

Una vez que hayas configurado tus preferencias, haz clic en el botón "Generate" para que Spring Initializr genere un proyecto base con las dependencias y configuraciones seleccionadas.

Paso 10

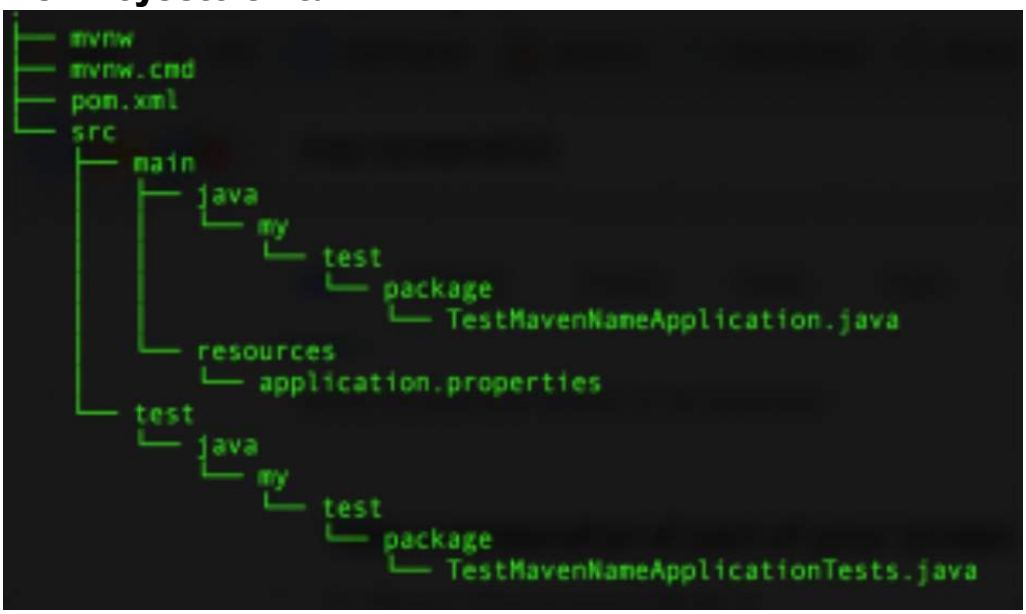
Descargar el Proyecto

Spring Initializr generará un archivo comprimido que contiene la estructura básica de tu proyecto. Descarga este archivo y guárdalo en una ubicación de tu elección en tu sistema local.

Paso 11

Importar el Proyecto en tu IDE

Abre el IDE (nuestra



recomendación es usar IntelliJ IDEA, pero podrías también usar el Spring Tool Suite de Eclipse) . Importa el proyecto desde el archivo comprimido descargado. En la mayoría de los casos, el IDE reconocerá automáticamente el tipo de proyecto y configurará las dependencias.

Paso 12

Comenzar a Desarrollar

Una vez que el proyecto esté importado en tu IDE, estarás listo para comenzar a desarrollar tu aplicación de Spring Boot. Puedes crear controladores, servicios y otros componentes directamente en el código Java, aprovechando las ventajas de la configuración automática y las opinionated defaults de Spring Boot.

Ventajas de Crear Proyectos con Spring Initializr

Rapidez

Generar un proyecto base con las dependencias correctas es rápido y evita la configuración manual.

Consistencia

Spring Initializr garantiza que tu proyecto esté configurado de acuerdo con las mejores prácticas y estándares de Spring Boot.

Configuración Predefinida:

Las opciones de dependencias predefinidas y las opinionated defaults de Spring Boot simplifican el desarrollo

Escalabilidad

Puedes agregar o eliminar dependencias según tus necesidades, lo que facilita la escalabilidad del proyecto.

Menos Errores

Al reducir la necesidad de configuraciones manuales, se minimiza la posibilidad de errores humanos.

¿Qué es IoC?

IoC es un principio de diseño que plantea que la responsabilidad de crear y administrar objetos y sus dependencias debe ser transferida a un contenedor o marco de trabajo. En otras palabras, en lugar de que las aplicaciones controlen la creación y gestión de sus componentes, como solía ser el enfoque tradicional, se delega esta responsabilidad a un sistema IoC… ¡el control se invierte!

La importancia de IoC

Desacoplamiento: promueve un bajo acoplamiento entre componentes. Los componentes no necesitan conocer los detalles de implementación de sus dependencias, lo que facilita su reutilización y pruebas unitarias.

•**Modularidad:** Al dividir una aplicación en componentes independientes con responsabilidades específicas, IoC fomenta la modularidad. Esto hace que el código sea más claro, mantenible y escalable.

•**Facilita las Pruebas:** La Inversión de Control simplifica las pruebas unitarias. Puedes reemplazar fácilmente las dependencias con objetos de prueba (mocks) para aislar las pruebas de una unidad específica de código.

•**Flexibilidad:** Con IoC, es más fácil cambiar implementaciones de componentes sin modificar el código de otros componentes. Esto es crucial en aplicaciones que evolucionan con el tiempo.

•**Mejora la Organización del Código:** Al separar la creación y gestión de componentes, tu código se vuelve más ordenado y coherente. La configuración se centraliza en un lugar, lo que facilita el mantenimiento.

•**Facilita la Integración:** IoC es esencial en entornos de desarrollo empresarial, donde las aplicaciones deben integrarse con diversas tecnologías y sistemas

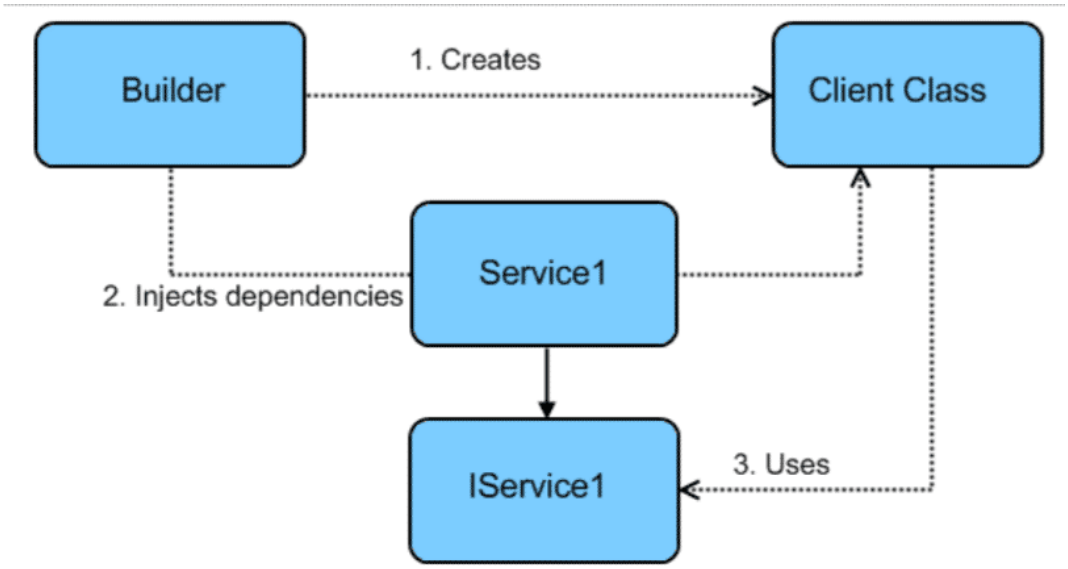
Si te quedaron dudas, te invitamos a ver el siguiente video: https://youtu.be/hlnDAy8lD00

Concepto de Dependency Injection

Ahora que sabemos lo que es la inversión de control, veamos el concepto de inyección de dependencia.

Dependency Injection (Inyección de Dependencias) es un patrón de diseño y un principio de desarrollo de software que se utiliza para lograr la Inversión de Control (IoC). Bajo este patrón, las dependencias de una clase son proporcionadas por fuera de la clase en lugar de ser creadas internamente. Esto significa que una clase no se encarga de crear las instancias de sus propias dependencias, sino que las recibe desde el exterior, lo que promueve un bajo acoplamiento y facilita la reutilización y pruebas unitarias.

En



términos simples, Dependency Injection es proporcionar los objetos necesarios (dependencias) a una clase en lugar de que la clase los cree por sí misma.

Elementos Clave de Dependency Injection:

Clase Dependiente

Es la clase que necesita una o más dependencias para realizar su trabajo. Esta clase no se encarga de crear estas dependencias.

Dependencias

Son objetos o instancias que la clase dependiente necesita para funcionar correctamente. Estas dependencias pueden ser otras clases, componentes, servicios o recursos.

Contenedor o Inyector:

Es un componente que se encarga de crear y administrar las dependencias y luego las inyecta en la clase dependiente. El contenedor resuelve cuáles dependencias se necesitan y proporciona instancias adecuadas.

Ventajas de Dependency Injection:

Desacoplamiento

La DI reduce el acoplamiento entre componentes de software, ya que las dependencias no están fuertemente acopladas a la clase que las utiliza. Esto facilita la reutilización de componentes y cambios en la implementación sin afectar otras partes del sistema.

Pruebas Unitarias:

Al recibir sus dependencias desde el exterior, las clases dependientes pueden ser probadas de manera más efectiva mediante la sustitución de las dependencias reales por objetos de prueba (mocks).

Flexibilidad

Cambiar o personalizar las implementaciones de las dependencias se vuelve más sencillo, ya que el contenedor puede ser configurado para proporcionar diferentes implementaciones sin modificar el código de la clase dependiente.

Tipos de DI: Constructor, Setter, y Field Injection

Existen tres maneras para construir las inyecciones.

Inyección por Constructor:

La Inyección por Constructor es un método en el que las dependencias requeridas se pasan como argumentos al constructor de la clase dependiente. Esto significa que cuando se crea una instancia de la clase dependiente, se deben proporcionar todas las dependencias necesarias en ese momento. Cabe mencionar, que es la forma recomendada para realizar inyección de dependencias en Spring.

```
public class ClaseDependiente {
    private Dependencia dependencia;
    public ClaseDependiente(Dependencia dependencia) {
        this.dependencia = dependencia;
    }
    public void realizarOperacion() {
        // Utiliza la dependencia para realizar una operación
        dependencia.operacion();
    }
}
```

Ventajas

- Asegura que las dependencias esenciales estén disponibles desde el inicio, lo que reduce la posibilidad de un estado inconsistente.
- Promueve la inmutabilidad, ya que las dependencias no se pueden cambiar después de la creación.

Desventajas

- Puede requerir constructores con muchos parámetros si hay muchas dependencias.

Inyección por Setter:

La inyección por Setter implica proporcionar métodos "setter" en la clase dependiente para asignar las dependencias requeridas. Esto permite cambiar las dependencias en cualquier momento después de la creación de la instancia. Esta es la forma menos recomendada de proceder.

```
public class ClaseDependiente {
    private Dependencia dependencia;
    public void setDependencia(Dependencia dependencia) {
        this.dependencia = dependencia;
    }
    public void realizarOperacion() {
        // Utiliza la dependencia para realizar una operación
        dependencia.operacion();
    }
}
```

Ventajas

- Permite la flexibilidad de cambiar las dependencias en tiempo de ejecución.
- Útil cuando algunas dependencias son opcionales y no es necesario proporcionarlas todas en la creación

Desventajas

- Puede llevar a un estado inconsistente si las dependencias no se establecen correctamente antes de usar la clase.

Inyección por propiedad:

La Inyección por propiedad o campo se basa en el uso de anotaciones, como @Autowired en Spring, para marcar campos en la clase dependiente. El contenedor de Spring se encarga de asignar las dependencias automáticamente.

```
public class ClaseDependiente {
    @Autowired
    private Dependencia dependencia;
    public void realizarOperacion() {
        // Utiliza la dependencia para realizar una operación
     dependencia.operacion();
    }
}
```

Ventajas

- Simplifica la configuración, ya que el contenedor se encarga de la asignación de dependencias.
- Menos código boilerplate en comparación con los métodos de constructor y setter.

Desventajas:

- Puede ser menos explícito en comparación con los métodos de constructor y setter.
- Requiere un contenedor de IoC, como Spring, para funcionar.

En general, la elección entre estos métodos depende de los requisitos específicos de tu aplicación. La Inyección por Constructor es una opción sólida cuando deseas garantizar que las dependencias esenciales estén disponibles desde el principio. La Inyección por Setter es útil cuando deseas flexibilidad para cambiar dependencias en tiempo de ejecución. La Inyección por Anotaciones es conveniente cuando deseas reducir la configuración manual y aprovechar las capacidades de un contenedor de IoC. Cada enfoque tiene sus ventajas y desventajas, y la elección dependerá de la situación y las necesidades de tu proyecto.

Aprende mas: https://www.juntadeandalucia.es/servicios/madeja/contenido/recurso/

142#:~:text=Spring%20provee%20tres%20tipos%20de,s%C3%B3lo%20utilizamos%20las%20dos%20primeras.&text=En%20este%20ejemplo%2C%20el%20bean,robotService%20usa%20inyecci%C3%B3n%20por%20setter.

Resolución de dependencias en Spring

La resolución de dependencias es una parte fundamental del funcionamiento de Spring Framework. Cuando trabajas con Spring, especialmente con Inyección de Dependencias (Dependency Injection), el marco se encarga de resolver y proporcionar las dependencias necesarias a las clases que las requieren. Aquí se explica cómo ocurre la resolución de dependencias en Spring:

- **Configuración del Contenedor de Spring:** En Spring, todas las configuraciones relacionadas con las dependencias se definen en un archivo de configuración (generalmente en formato XML o utilizando anotaciones en Spring Boot como ya vimos en una lección anterior). Esto define cómo se crean y gestionan los componentes y sus relaciones.
- **Definición de Componentes:** En el archivo de configuración o a través de las anotaciones de Spring Boot, se definen los componentes de la aplicación, como servicios, repositorios o cualquier otra clase que necesite ser administrada por Spring. Estas clases se definen como beans en el contexto de Spring.
- **Identificación de Dependencias:** Cuando una clase necesita una dependencia, no la crea directamente ni la busca por sí misma. En su lugar, la clase declara que requiere una dependencia y Spring se encarga de proporcionarla.
- **Inyección de Dependencias:** La resolución de dependencias ocurre cuando Spring inyecta las dependencias requeridas en la clase correspondiente. Esto se puede hacer a través de un constructor (Inyección por Constructor), métodos "setter" (Inyección por Setter), o incluso mediante anotaciones (Inyección por Campo).
- **Ciclo de Vida del Bean:** Spring gestiona el ciclo de vida de los beans. Cuando una dependencia se inyecta en una clase, Spring asegura que la instancia sea la correcta y esté en el estado adecuado (por ejemplo, una conexión a la base de datos está abierta) antes de entregarla.
- **Alcance (Scope):** Spring permite definir el alcance de los beans, lo que controla cuántas instancias de una clase se crean y cuánto tiempo persisten. Los alcances comunes son "singleton" (una única instancia) y "prototype" (nueva instancia cada vez).
- **Resolución de Dependencias en Tiempo de Ejecución:** Spring realiza la resolución de dependencias en tiempo de ejecución, lo que significa que las dependencias se asignan cuando la aplicación se está ejecutando, no en tiempo de compilación.

Uso de anotaciones para realizar la inyección

Pero claro, nosotros estamos utilizando Spring Boot así que, ¿por qué no delegar todo esto?

En Spring Boot, como ya vimos, puedes utilizar anotaciones para simplificar la configuración y realizar la DI de manera más concisa. Estas anotaciones se utilizan en lugar de la configuración basada en XML y permiten que Spring escanee automáticamente tu código en busca de componentes y realice la inyección de dependencias de manera automática. Aquí hay algunas de las anotaciones más comunes utilizadas en Spring:

@Component:

Esta anotación se utiliza para marcar una clase como un componente de Spring. Spring escaneará el paquete actual y sus subpaquetes en busca de clases marcadas con @Component y las registrará como beans en el contexto de Spring. Por ejemplo:

```
@Component
public class MiComponente {
    // ...
}
```

@Service:

Esta anotación se utiliza específicamente para marcar clases de servicios en Spring. Es una especialización de @Component que otorga un nivel semántico a nuestra clase. Los servicios suelen contener la lógica empresarial de la aplicación. Por ejemplo:

```
@Service
public class MiServicio {
    // ...
}
```

@Repository:

Utilizada para marcar clases de acceso a datos (por ejemplo, clases de repositorio de Spring Data). Al igual que @Service, es una especialización semántica de @Component. Por ejemplo:

```
@Repository
public class MiRepositorio {
    // ...
}
```

@Autowired:

Esta anotación se utiliza para realizar la inyección de dependencias en un campo, constructor o método setter. Spring busca una instancia del tipo requerido y la inyecta automáticamente en el lugar marcado con @Autowired. Por ejemplo:

```
@Service
public class MiServicio {
    private MiRepositorio repositorio;

    @Autowired
    public MiServicio(MiRepositorio repositorio) {
        this.repositorio = repositorio;
    }
}
```

@Qualifier:

Se utiliza junto con @Autowired para especificar cuál bean específico debe inyectarse cuando hay múltiples beans del mismo tipo. Puedes usar el nombre del bean o el valor de la anotación @Qualifier para seleccionar el bean adecuado.

@Value:

Utilizada para inyectar valores desde propiedades o archivos de configuración en campos de una clase. Esto es útil para configurar valores como rutas de archivos, números, cadenas, etc.

@Configuration:

Marca una clase como una clase de configuración de Spring. Se utiliza en lugar de archivos XML para definir la configuración de Spring. Los métodos dentro de esta clase que están marcados con @Bean se utilizan para definir beans de Spring.

Estas son algunas de las anotaciones más utilizadas en Spring para la inyección de dependencias y la configuración. Al utilizar estas anotaciones de manera adecuada, puedes hacer que tu código sea más claro y conciso. Spring se encargará de la resolución de dependencias y la administración de los componentes marcados automáticamente.

Ciclo de vida de los Beans y el patrón Singleton

Como ya vimos, Spring Framework es una herramienta que facilita la creación y gestión de componentes.
Una de las piedras angulares de éste es el concepto de "Bean". En esta clase, exploramos a profundidad qué es un Bean en Spring y cómo se relaciona con el ciclo de vida de una aplicación.
Los Beans de Spring son más que simplemente objetos; representan las piezas fundamentales de una aplicación, desde controladores y servicios hasta componentes de acceso a datos y más. Comprender qué es un Bean y cómo se gestiona su ciclo de vida es esencial para desarrollar aplicaciones robustas y mantenibles.
En las **próximas secciones,** explicaremos el ciclo de vida de un Bean en Spring, desde su creación hasta su destrucción, y cómo podemos personalizar este proceso. También discutiremos el uso de interfaces y métodos para influir en el ciclo de vida de los Beans y cómo los eventos en el ciclo de vida pueden ser cruciales para la gestión y el monitoreo de nuestras aplicaciones. En última instancia, aprenderemos cómo podemos tomar el control total del ciclo de vida de los Beans para adaptarlo a las necesidades específicas de nuestras aplicaciones.
Sigue leyendo para descubrir cómo **Spring framework** hace que el desarrollo de aplicaciones empresariales sea más eficiente y poderoso a través de la gestión de Beans y su ciclo de vida.

En el contexto de Spring Framework, el término "Bean" se utiliza para referirse a un objeto o componente gestionado por el contenedor de Spring.

Un Bean es una instancia de una clase que es administrada por Spring y está configurada para ser utilizada en una aplicación. Estas clases pueden ser desde simples objetos hasta componentes más complejos, como servicios, controladores web, repositorios de datos, entre otros.
La principal característica de un Bean es que su ciclo de vida, creación y configuración están bajo el control de Spring, lo que significa que Spring se encarga de la creación de instancias, la inyección de dependencias y la configuración de propiedades. Esto proporciona varios beneficios, incluyendo:

- **Inversión de Control (IoC):** Spring invierte el control sobre la creación y gestión de objetos, lo que reduce la dependencia de la lógica de la aplicación en la creación de instancias.

- **Inyección de Dependencias:** Los Beans pueden depender de otros Beans, y Spring se encarga de inyectar estas dependencias de manera automática.
- **Configuración Externa:** Los Beans pueden configurarse externamente, generalmente a través de archivos XML o anotaciones (es el caso de Spring Boot), lo que facilita la configuración y ajuste de la aplicación sin modificar el código fuente.
- **Gestión de Ciclo de Vida:** Spring controla el ciclo de vida de los Beans, permitiendo acciones específicas en diferentes etapas, como inicialización y destrucción.
- **Reutilización y Pruebas Unitarias:** Los Beans son fácilmente reutilizables y simplifican las pruebas unitarias al permitir la sustitución de dependencias reales por objetos de prueba.

Ve el siguiente video donde se explica el concepto de BEAN en Spring Boot: https://youtu.be/ZCy8JzmZi8Q

Ciclo de vida de un Bean en Spring

Una vez definido que es Bean, veamos su ciclo de vida. El ciclo de vida de un Bean en Spring es un concepto esencial para comprender cómo Spring administra los componentes dentro de una aplicación. Cada Bean pasa por una serie de etapas desde su creación hasta su eventual destrucción. Estas etapas permiten que Spring realice configuraciones, inyecciones de dependencias y acciones personalizadas según sea necesario.

A continuación, se describen las etapas clave del ciclo de vida de un Bean en Spring:

Instanciación

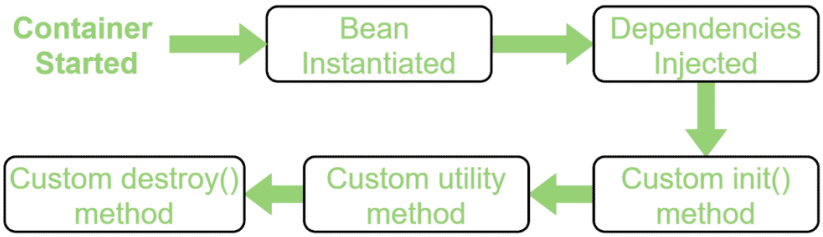
En esta primera etapa, Spring crea una instancia del Bean. Todos los Beans son almacenados en un gran contenedor (el Spring Container) que se convierte en el “contexto”, de dónde luego irá tomando los Beans según se requiera. Esto generalmente implica la creación de un objeto utilizando un constructor o un método de fábrica.

Inyección de Dependencias

Una vez que se crea la instancia del Bean, Spring inyecta las dependencias requeridas. Esto se logra mediante la Inyección de Dependencias, donde Spring proporciona las dependencias definidas en el archivo de configuración o mediante anotaciones.

Post Procesamiento (opcional)

Spring permite que los Beans realicen acciones adicionales después de la inyección de dependencias, si es necesario. Esto se puede lograr implementando interfaces como BeanPostProcessor para personalizar el proceso de inicialización.



Inicialización
En esta etapa, Spring invoca métodos de inicialización personalizados si están definidos en el Bean. Puedes usar la anotación @PostConstruct o un método personalizado de inicialización para realizar configuraciones específicas antes de que el Bean esté completamente disponible.
Uso del Bean
Una vez que se ha inicializado, el Bean está listo para su uso en la aplicación. Las clases que dependen de este Bean pueden utilizarlo para llevar a cabo sus tareas específicas.
Destrucción (opcional):
Al final de la vida útil de un Bean, Spring puede invocar métodos de destrucción personalizados, si están definidos. Esto se puede lograr utilizando la anotación @PreDestroy o métodos personalizados de destrucción. La destrucción se realiza para liberar recursos y realizar limpieza cuando ya no se necesita el Bean.
Finalización
El Bean se elimina finalmente del contexto de Spring y queda fuera de alcance para su uso. En esta etapa, se liberan los recursos asociados con el Bean.

Comprender el ciclo de vida de un **Bean** en Spring es crucial para aprovechar al máximo el framework. Te permite controlar cuándo y cómo se realizan las configuraciones y personalizaciones, y garantiza que los Beans estén disponibles y configurados correctamente durante toda la vida de tu aplicación. Esto promueve la modularidad, la reutilización y la facilidad de mantenimiento en tus proyectos de desarrollo empresarial.

Uso de interfaces y métodos para el ciclo de vida

En Spring Framework, tienes la capacidad de personalizar el ciclo de vida de un Bean utilizando interfaces y métodos específicos. Esto te permite ejecutar código personalizado en momentos cruciales del ciclo de vida del Bean. Aquí se describen

dos interfaces clave y los métodos relacionados que puedes implementar para esta personalización:

1. InitializingBean e Initializing Method:

- InitializingBean: Es una interfaz proporcionada por Spring que contiene un método llamado `afterPropertiesSet()`. Cuando una clase implementa esta interfaz, Spring automáticamente invoca el método `afterPropertiesSet()` después de que se hayan establecido todas las propiedades y se haya completado la inyección de dependencias en el Bean.
- Método de Inicialización Personalizado: Además de implementar la interfaz `InitializingBean`, puedes definir un método personalizado de inicialización en tu clase Bean. Este método puede llevar cualquier nombre, pero debe ser anotado con `@PostConstruct`. Spring invocará este método después de `afterPropertiesSet()` si ambos están presentes.

Ejemplo de implementación:

```
import org.springframework.beans.factory.InitializingBean;
import javax.annotation.PostConstruct;

public class MiBean implements InitializingBean {

    // Método definido por InitializingBean
    @Override
    public void afterPropertiesSet() throws Exception {
        // Lógica de inicialización personalizada
    }

    // Método de inicialización personalizado
    @PostConstruct
    public void init() {
        // Lógica de inicialización personalizada
    }
}
```

2. DisposableBean e Inicialización Personalizada:

- **DisposableBean:** Al igual que con la interfaz `InitializingBean`, Spring proporciona la interfaz `DisposableBean`, que contiene el método `destroy()`. Este método se llama cuando Spring destruye el Bean, permitiéndote realizar cualquier limpieza necesaria.
- **Método de Destrucción Personalizado**: Además de implementar la interfaz `DisposableBean`, puedes definir un método personalizado de destrucción en tu Bean. Debes anotarlo con `@PreDestroy`. Spring invocará este método antes de `destroy()` si ambos están presentes.

•Ejemplo de implementación:

```
import org.springframework.beans.factory.DisposableBean;
import javax.annotation.PreDestroy;

public class MiBean implements DisposableBean {
    // Método definido por DisposableBean
    @Override
```

```
    public void destroy() throws Exception {
        // Lógica de destrucción personalizada
    }

    // Método de destrucción personalizado
    @PreDestroy
    public void customDestroy() {
        // Lógica de destrucción personalizada
    }
}
```

Estas interfaces y métodos personalizados brindan el control sobre las acciones que ocurren durante la inicialización y la destrucción de tus **Beans** en Spring. Puedes utilizarlos para realizar configuraciones específicas o liberar recursos cuando sea necesario. Esto es especialmente útil en situaciones donde los Beans tienen dependencias externas o requieren acciones especiales antes o después de su uso.

Eventos en el ciclo de vida de los Beans

En Spring Framework, los eventos juegan un papel crucial en la gestión de componentes y permiten que los Beans interactúen y reaccionen a eventos específicos durante su ciclo de vida. Estos eventos son una parte esencial de la arquitectura de Spring y proporcionan un mecanismo poderoso para la comunicación y la personalización en una aplicación. Aquí se exploran los eventos en el ciclo de vida de los Beans en Spring:

1. Eventos de Contexto de Aplicación:

- ***ContextRefreshedEvent:*** Este evento se desencadena cuando se refresca el contexto de la aplicación, lo que significa que la aplicación está lista y todos los Beans han sido inicializados.
- ***ContextStartedEvent:*** Se genera cuando se inicia el contexto de la aplicación. Indica que la aplicación está en funcionamiento.
- ***ContextStoppedEvent:*** Este evento ocurre cuando se detiene el contexto de la aplicación. Indica que la aplicación se ha detenido.
- ***ContextClosedEvent:*** Se desencadena cuando se cierra el contexto de la aplicación, generalmente cuando se apaga la aplicación. Indica que la aplicación se está cerrando.

2. Eventos de Bean

- *InitializationEvent:* Este evento se genera después de que un Bean se ha inicializado y antes de que esté disponible para su uso. Es útil para realizar acciones inmediatamente después de la inicialización de un Bean.
- ***DestructionEvent***: Este evento se desencadena antes de que se destruya un Bean, lo que proporciona la oportunidad de realizar limpieza o liberación de recursos antes de que el Bean se elimine.

3. Eventos Personalizados:

Además de los eventos predefinidos, Spring te permite crear y publicar tus propios eventos personalizados. Puedes definir clases de eventos personalizados y publicarlos en el contexto de la aplicación cuando sea necesario. Esto es útil para notificar eventos específicos dentro de tu aplicación y permitir que los Beans reaccionen en consecuencia.
Para publicar un evento personalizado, puedes utilizar la clase ApplicationEventPublisher, que está disponible en el contexto de la aplicación de Spring.

Beneficios de los Eventos en Spring:

- **Desacoplamiento**: Los eventos permiten que los componentes de la aplicación se comuniquen sin estar acoplados directamente. Los productores de eventos no necesitan conocer a los consumidores y viceversa.
- **Extensibilidad**: Puedes crear eventos personalizados para modelar situaciones específicas de tu aplicación y permitir una respuesta personalizada
- **Flexibilidad**: Los eventos brindan un mecanismo para la coordinación y la gestión de componentes en una aplicación, lo que facilita la implementación de patrones de diseño como el Observador.

En resumen, los eventos en el ciclo de vida de los Beans en Spring son una herramienta poderosa para la comunicación y la personalización en aplicaciones empresariales. Estos eventos proporcionan un nivel adicional de flexibilidad y modularidad, lo que facilita la creación de aplicaciones más robustas y mantenibles.

Para aprender mas:
https://inmensia.com/articulos/spring/beans_ciclos_de_vida.html#:~:text=El%20ciclo%20de%20vida%20de,a%20trav%C3%A9s%20de%20m%C3%A9todos%20callback.

Sobre patrones de diseño en Spring

Para comenzar, vamos a explorar brevemente algunos de los patrones que implementa (o puede implementar) Spring de fondo para gestionar beans en una aplicación, y cómo Spring los utiliza para mejorar el desarrollo de aplicaciones.

`@Patrón Factory:`

El patrón Factory se trata de crear objetos sin especificar la clase exacta de objeto que se creará. En Spring, este patrón se implementa mediante el uso de fábricas de beans, donde el contenedor de Spring es responsable de crear y administrar los objetos. Spring ofrece diversas formas de configurar estas fábricas, como las anotaciones `@Component`, `@Service`, `@Repository` y `@Controller`. Estas anotaciones permiten que Spring genere automáticamente instancias de clases y las administre como beans.

@Component

```
public class UserServiceFactory {
    public UserService createUserService(String userType) {
        if ("admin".equals(userType)) {
            return new AdminUserService();
        } else if ("regular".equals(userType)) {
            return new RegularUserService();
```

```
        } else {
            throw new IllegalArgumentException("Invalid user type");
        }
    }
}

public interface UserService {
    void processUser();
}

@Component
public class AdminUserService implements UserService {
    @Override
    public void processUser() {
        System.out.println("Processing admin user");
    }
}

@Component
public class RegularUserService implements UserService {
    @Override
    public void processUser() {
        System.out.println("Processing regular user");
    }
}
```

Patrón Observer:

El **patrón Observer** se refiere a establecer una relación uno a muchos entre objetos, de manera que cuando un objeto cambia de estado, todos los objetos dependientes sean notificados y actualizados automáticamente. En Spring, este patrón se implementa a través de eventos y listeners.

Spring proporciona un modelo de eventos que permite a los componentes comunicarse de manera desacoplada. Los objetos pueden publicar eventos y otros objetos, conocidos como listeners, pueden suscribirse a esos eventos y actuar en consecuencia. Esto es útil para la comunicación entre componentes en una aplicación modular.

@Component

```
public class EventPublisher {
    @Autowired
    private ApplicationEventPublisher eventPublisher;
    public void publishEvent(String message) {
        eventPublisher.publishEvent(new CustomEvent(this, message));
    }}

@Component
public class CustomEvent extends ApplicationEvent {
    private String message;
    public CustomEvent(Object source, String message) {
        super(source);
        this.message = message;
    }

    public String getMessage() {
        return message;
    }
}

@Component
public class EventListener {
```

```
    @EventListener
    public void handleCustomEvent(CustomEvent event) {
        System.out.println("Received custom event: " + event.getMessage());
    }
}
```

`Patrón Strategy:`

El patrón Strategy implica definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables. En Spring, este patrón se implementa mediante la inyección de dependencias y la configuración de beans.

Esto le permite permite definir varios componentes que implementan diferentes estrategias y luego inyectar la estrategia adecuada en un componente cliente. Esto permite cambiar el comportamiento de una aplicación sin modificar el código cliente.

```
    public interface PaymentStrategy {
    void processPayment(double amount);
}

@Component
public class CreditCardPaymentStrategy implements PaymentStrategy {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}

@Component
public class PayPalPaymentStrategy implements PaymentStrategy {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}


@Component
public class PaymentProcessor {
    private PaymentStrategy paymentStrategy;

    @Autowired
    public PaymentProcessor(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }

    public void processOrder(double amount) {
        paymentStrategy.processPayment(amount);
    }
}
```

@Patrón Template Method:

El patrón Template Method define la estructura de un algoritmo en una clase base, permitiendo que las subclases implementen pasos específicos. En Spring, este patrón se utiliza en el diseño de plantillas de procesamiento.

Spring ofrece plantillas para diferentes tareas, como acceso a bases de datos, manipulación de transacciones y envío de correos electrónicos. Las plantillas definen la

estructura general del proceso, mientras que las implementaciones específicas se proporcionan en las subclases.

@Component

```
public abstract class DataProcessorTemplate {
    public void processData(String data) {
        validateData(data);
        transformData(data);
        storeData(data);
    }

    protected abstract void validateData(String data);
    protected abstract void transformData(String data);
    protected abstract void storeData(String data);
}

@Component
public class CsvDataProcessor extends DataProcessorTemplate {
    @Override
    protected void validateData(String data) {
        System.out.println("Validating CSV data: " + data);
    }

    @Override
    protected void transformData(String data) {
        System.out.println("Transforming CSV data: " + data);
    }

    @Override
    protected void storeData(String data) {
        System.out.println("Storing CSV data: " + data);
    }
}
```

Patrón Dependency Injection (Inyección de Dependencias):

Como ya vimos, si bien no es un patrón clásico, es fundamental en Spring. Implica pasar las dependencias necesarias a un objeto en lugar de que el objeto las cree internamente. Ya sabemos que Spring es conocido por su enfoque en la inversión de control y la inyección de dependencias, permitiendo definir las dependencias entre componentes en un archivo de configuración XML o mediante anotaciones. Luego, inyecta automáticamente las dependencias en los componentes correspondientes. Esto promueve la modularidad, el desacoplamiento y facilita las pruebas unitarias.

Estos son solo algunos ejemplos de cómo Spring utiliza diferentes patrones de diseño para mejorar el diseño de las aplicaciones. Cada patrón aborda problemas específicos y proporciona soluciones probadas para garantizar la modularidad, la reutilización y la mantenibilidad del código.

¿Y el patrón Singleton? El patrón por defecto en Spring

Veamos cómo Spring aprovecha el patrón Singleton en el núcleo de su diseño:

Ámbito Predeterminado

En Spring, los Beans (componentes administrados por Spring) siguen el ámbito Singleton de manera predeterminada. Esto significa que Spring crea una única instancia de un Bean y la reutiliza en toda la aplicación. Esto garantiza eficiencia y consistencia en el uso de recursos compartidos.

Centralización de Gestión

Spring actúa como un administrador centralizado de componentes (Beans). Esto facilita el acceso a instancias Singleton desde cualquier parte de la aplicación, lo que promueve la colaboración y el mantenimiento eficiente.

Configuración Declarativa:

Spring permite configurar Beans Singleton de manera declarativa, lo que significa que puedes definir sus características a través de anotaciones, archivos XML o código Java. Esta configuración sencilla reduce la complejidad de la gestión manual de instancias Singleton

Promoción de Inyección de Dependencias:

Spring impulsa el uso de la Inyección de Dependencias (DI). Esto te permite inyectar dependencias en tus Beans Singleton, lo que mejora la modularidad y facilita las pruebas unitarias al proporcionar una forma sencilla de reemplazar dependencias durante las pruebas.

Ventajas y desventajas del patrón Singleton

El patrón Singleton es ampliamente utilizado en la programación orientada a objetos, pero como cualquier diseño, tiene sus ventajas y desventajas. Acá vemos algunas de las principales:

Ventajas del Patrón Singleton:

- **Única Instancia**: Garantiza que una clase tenga una única instancia en toda la aplicación. Esto es útil cuando se necesita un punto centralizado para coordinar acciones o compartir recursos.
- **Acceso Global:** Proporciona un punto de acceso global a la instancia única, lo que facilita la comunicación y la gestión de recursos compartidos.
- **Eficiencia:** Al evitar la creación repetida de instancias, el patrón Singleton ahorra recursos de memoria y tiempo de inicialización, lo que puede ser crítico en aplicaciones con recursos limitados.
- **Control de Acceso:** Permite controlar el acceso a la instancia única, lo que puede ser útil para aplicar reglas específicas o garantizar la coherencia de los datos compartidos.
- **Facilita Pruebas Unitarias**: Al proporcionar un punto de acceso global, Singleton simplifica la inyección de dependencias y la sustitución de implementaciones durante las pruebas unitarias.

Desventajas del Patrón Singleton:

- **Acoplamiento Fuerte:** El uso excesivo del patrón Singleton puede crear un alto acoplamiento entre clases, lo que puede dificultar la modularidad y el mantenimiento.
- **Dificultad en Pruebas de Unidad:** Si una clase Singleton depende directamente de otros Singleton, las pruebas unitarias pueden volverse complicadas, ya que puede ser difícil sustituir esas dependencias durante las pruebas.
- **Problemas en Entornos Multihilo:** En entornos multihilo, la creación y el acceso a la instancia Singleton deben ser seguros para evitar problemas de concurrencia. Esto puede aumentar la complejidad del código.
- **Potencial para Anti-Patrones:** Si se utiliza en exceso, el patrón Singleton puede llevar a anti-patrones como el "Dios Singleton", donde un solo Singleton contiene demasiada lógica y se convierte en un punto de falla crítico.
- **Dificultad en Herencia**: En algunos casos, puede ser complicado permitir la herencia de una clase Singleton, lo que puede limitar la flexibilidad de diseño.

Implementación del patrón Singleton en Spring

En Spring, la implementación del patrón Singleton es la configuración predeterminada para los Beans (componentes administrados por Spring). Spring garantiza que solo haya una instancia de un Bean en todo el contexto de la aplicación por defecto. Esto significa que, cuando defines un Bean en Spring, obtienes automáticamente un Singleton.
El siguiente es un ejemplo de cómo se define en código, la lógica de una clase que implemente el patrón Singleton:

```
public class MiSingleton {
    private static MiSingleton instance;

    // Constructor privado para evitar instanciación externa
    private MiSingleton() {
        // Inicialización
    }

    public static MiSingleton getInstance() {
        if (instance == null) {
            instance = new MiSingleton();
        }
        return instance;
    }

    // Métodos de la clase
}
```

La clase Singleton tiene un constructor privado para evitar que se cree una instancia directamente desde fuera de la clase. El método getInstance() verifica si ya existe una instancia de Singleton. Si no existe, crea una nueva instancia y la

almacena en la variable estática instance. Si ya existe, simplemente devuelve la instancia existente.
Puedes acceder a esta instancia Singleton desde cualquier parte de tu aplicación utilizando Inyección de Dependencias, ya sea mediante anotaciones, XML o código Java. Por ejemplo:

```
@Service
public class MiServicio {

    private final MiSingleton singletonBean;

    @Autowired
    public MiServicio(MiSingleton singletonBean) {
        this.singletonBean = singletonBean;
    }

    public void realizarOperacion() {
        singletonBean.saludar();
    }
}
```

En este servicio, hemos inyectado la instancia de **`MiSingleton`** como una dependencia. Como resultado, obtendrás la misma instancia de **`MiSingleton`** cada vez que inyectes este servicio en cualquier parte de tu aplicación.
Cabe recordar que no necesitas implementar el patrón Singleton manualmente. Spring gestiona automáticamente la creación y administración de instancias Singleton para los beans definidos en el contexto de Spring.
En resumen, Spring hace que la implementación del patrón Singleton sea sencilla y efectiva. Al definir tus Beans como componentes administrados por Spring, automáticamente obtienes el comportamiento Singleton sin tener que preocuparte por crear o gestionar manualmente las instancias únicas de tus clases.
En este video podemos ver un ejemplo más a fondo https://youtu.be/ZMmw_aZT76c

Consideraciones y buenas prácticas

Cuando implementas el patrón Singleton en Spring, es importante seguir algunas consideraciones y buenas prácticas para garantizar que tu aplicación sea eficiente y mantenible:

- **Usa la Configuración de Spring:** En lugar de utilizar la palabra clave new para crear instancias de tus clases, utiliza Spring para administrar la creación de instancias Singleton. Define tus Beans como componentes de Spring mediante anotaciones como @Component, @Service, @Repository, o configúralos en un archivo XML de configuración.
- **No uses Singletons para Datos Mutables**: Evita utilizar el patrón Singleton para clases que contienen datos mutables compartidos por múltiples partes de la aplicación, especialmente en entornos multihilo. Si es necesario compartir datos mutables, considera otras estrategias de concurrencia.

- **Ten en cuenta el Ámbito (scope) por Defecto**: En Spring, el ámbito por defecto de un Bean es Singleton. No necesitas especificarlo explícitamente a menos que quieras cambiar el ámbito. Siempre revisa y comprende el ámbito de tus Beans para evitar sorpresas inesperadas.
- **Evita Dependencias Cíclicas:** Evita las dependencias cíclicas entre tus Beans Singleton, ya que pueden llevar a bloqueos en la inicialización de la aplicación. Spring puede detectar dependencias circulares y lanzar una excepción al intentar crear el contexto de la aplicación.
- **Usa Inyección de Dependencias**: En lugar de acceder a Singleton Beans directamente mediante la palabra clave new o el uso de getInstance(), utiliza la Inyección de Dependencias de Spring para acceder a los Beans Singleton. Esto simplifica la gestión de dependencias y facilita las pruebas unitarias.
- **Sé Cauteloso con la Lógica en el Constructor:** Limita la lógica compleja en el constructor de tus Beans Singleton. El constructor debe ser lo más liviano posible para evitar problemas de rendimiento durante la inicialización.
- **Considera Usar Propiedades Inmutables:** Si es posible, define las propiedades de tus Beans Singleton como inmutables. Esto mejora la seguridad en entornos multihilo y evita cambios inesperados en las propiedades compartidas.
- **Realiza Pruebas Unitarias**: Asegúrate de probar tus Beans Singleton utilizando pruebas unitarias. Spring proporciona herramientas para facilitar las pruebas, como el uso de anotaciones como @Autowired y la capacidad de configurar Beans con valores específicos para las pruebas.
- **Documenta Uso Especial:** Si tienes Beans Singleton que se utilizan de manera especial o que deben ser compartidos entre múltiples componentes, documenta claramente su uso y propósito para otros desarrolladores.
- **Mantén la Modularidad:** Intenta mantener la modularidad de tu aplicación. Los Singletons deben utilizarse cuando sea necesario, pero no en exceso, ya que pueden aumentar el acoplamiento entre componentes.

Siguiendo estas consideraciones y buenas prácticas, puedes aprovechar al máximo la implementación del patrón Singleton en Spring y garantizar que tu aplicación sea eficiente y mantenible.

Creando una API RESTful con Spring Boot

¿Qué es REST (Representational State Transfer?

En su esencia, REST es un conjunto de principios arquitectónicos que definen cómo deben funcionar las interacciones entre sistemas distribuidos, como aplicaciones web y servicios en la nube. Estos principios se basan en la simplicidad, la escalabilidad y la interoperabilidad, lo que lo convierte en un enfoque popular en el desarrollo de APIs.

Principios Clave de REST:

Recursos

En una API RESTful, los recursos son las entidades que se pueden acceder y manipular a través de la API. Estos recursos pueden ser objetos de datos, como usuarios, productos o publicaciones, y se identifican mediante URLs únicas. Por ejemplo, un recurso de usuario podría tener la URL "/usuarios/123".

Verbos HTTP

Los verbos HTTP estándar, como GET, POST, PUT y DELETE, se utilizan para realizar operaciones en los recursos. Por ejemplo, una solicitud GET a "/usuarios/123" podría recuperar los detalles de un usuario, mientras que una solicitud POST a "/usuarios" podría crear un nuevo usuario.

Estado Representacional

En lugar de enviar objetos complejos, las solicitudes y respuestas de una API RESTful utilizan representaciones de estado de recursos. Estas representaciones suelen estar en formato JSON o XML y contienen información sobre el recurso en un momento dado.

Sin Estado

La arquitectura REST es sin estado, lo que significa que cada solicitud entre cliente y servidor debe contener toda la información necesaria para comprender y procesar la solicitud. El servidor no guarda información sobre el estado del cliente entre solicitudes, lo que facilita la escalabilidad.

Interfaz Uniforme

REST promueve una interfaz uniforme para acceder y manipular recursos. Esto incluye el uso de URL para identificar recursos, verbos HTTP estándar y la capacidad de navegar y descubrir recursos mediante enlaces.

Independencia de Capa

Los sistemas RESTful son independientes de la capa de almacenamiento subyacente. Esto significa que los recursos pueden estar almacenados en bases de datos, sistemas de archivos, servicios externos, etc.

En resumen, una **API RESTful** es una **API** que sigue los principios de la arquitectura REST. Utiliza HTTP para comunicarse y se basa en la idea de que todo es un recurso accesible a través de URLs, lo que facilita la construcción de

servicios web escalables y flexibles. Estas API se utilizan comúnmente en el desarrollo web y móvil para permitir la comunicación y el intercambio de datos entre aplicaciones y servicios.

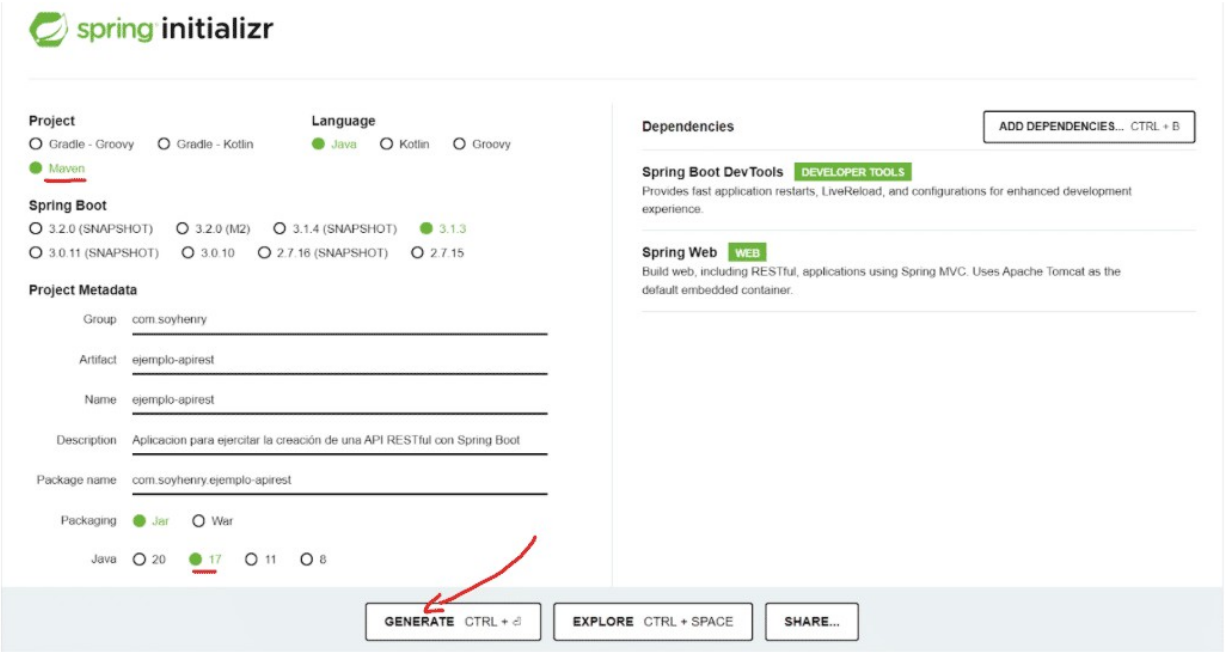
Manos a la obra con Spring Boot

A continuación, vamos a ponernos manos a la obra y hacer, paso a paso, una API RESTful utilizando Spring Boot!

Paso 1: Generar el proyecto con Spring Initializr

Abre tu navegador web y ve a https://start.spring.io/. Aquí puedes guiarte del mismo proceso que hicimos en la lección 2 para iniciar un proyecto, llena la información básica del proyecto, como el nombre del grupo, el nombre del artefacto y la descripción. Ten en cuenta que, para el ejemplo, utilizaremos Maven como gestor de dependencias y la versión 17 del SDK de Java.

En la sección de "Dependencies", busca y agrega las siguientes dependencias: "Spring Web" y "Spring Boot DevTools". Estas son importantes para crear una API RESTful y agilizar el desarrollo.

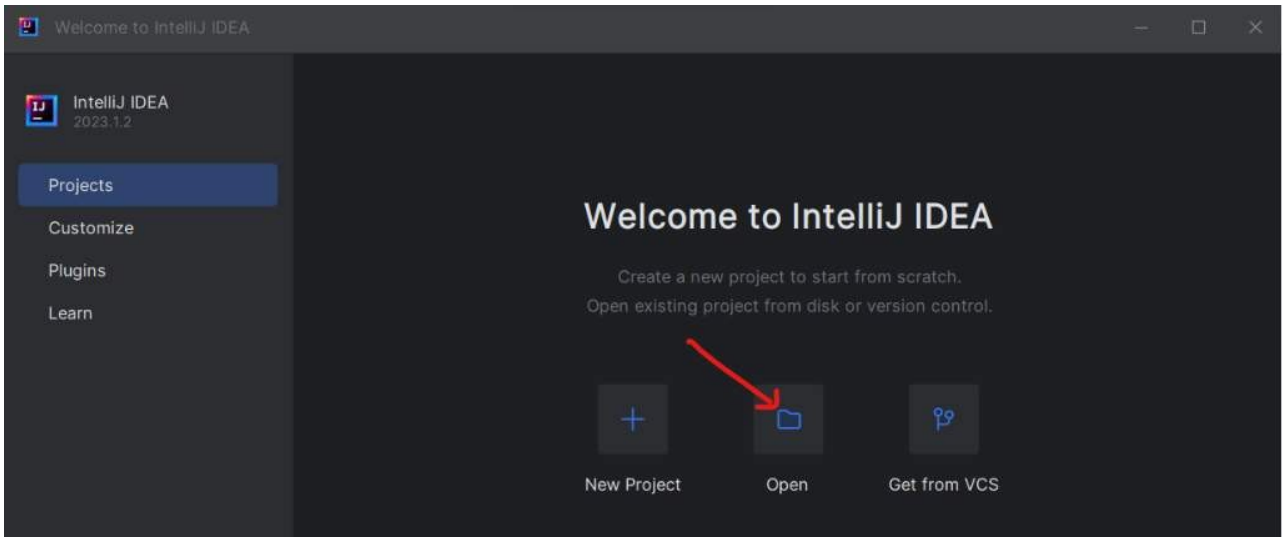


Haz clic en el botón "Generate" para descargar el archivo ZIP del proyecto generado.

Paso 2: Descomprimir y abrir el proyecto con IntelliJ IDEA

Descomprime el archivo ZIP descargado en una ubicación de tu elección.

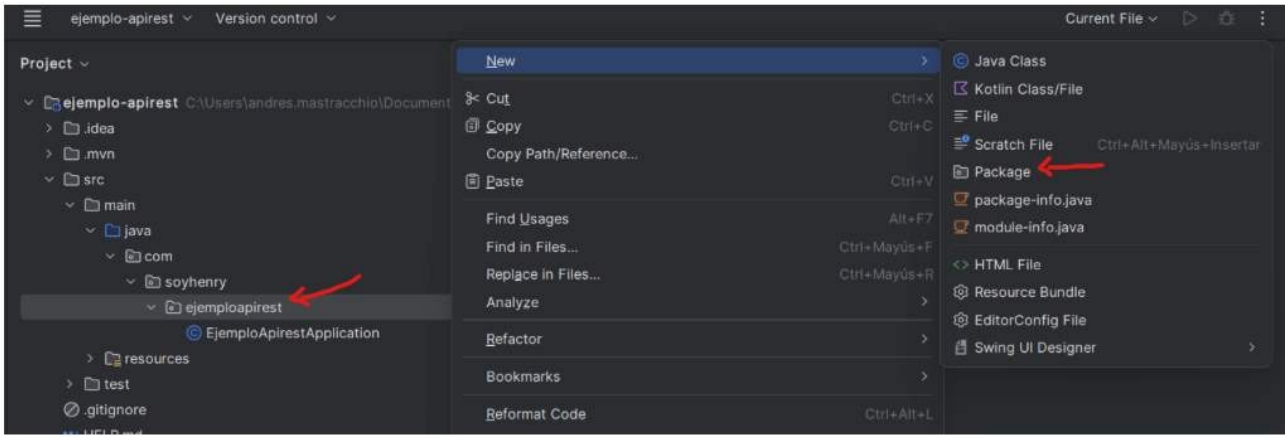
Abre IntelliJ IDEA y selecciona "Open" en el menú principal.



Navega hasta la carpeta del proyecto descomprimido y selecciónala. Luego, haz clic en "Open" para cargar el proyecto en IntelliJ.

Paso 3: Crear un Controller con endpoints POST y GET

En la carpeta principal, donde ahora se encuentra la clase de application, crea un paquete nuevo llamado “controller”.



Dentro de este paquete, crea una clase llamada "ApiController".

Abre "ApiController.java" y agrega la anotación `@RestController` encima de la clase para indicar que es un controlador REST.

También puedes agregar la anotación ‘@RequestMapping’ en la cual puedes especificar un path base para la gestión de esos elementos en dicho controlador (en el caso que la aplicación contenga más de un controlador).

Recuerda que luego, el path completo de la url se arma con ‘http://localhost:8080’ cuando inicias la aplicación de Spring Boot.

```
import org.springframework.web.bind.annotation.RestController;
@RestController
@RequestMapping(“/api/v1/elemento”)
public class ApiController {

}
```

Crea dos métodos en esta clase, uno para el endpoint POST y otro para el endpoint GET. Agrega las anotaciones `@PostMapping` y `@GetMapping` respectivamente. También,

asegúrate de importar las clases o interfaces necesarias como List o ResponseEntity. Ten en cuenta que, por el momento, no hemos creado la entidad ‘Elemento’.

```
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.GetMapping;

@RestController
public class ApiController {
    @PostMapping("/crear")
        public  ResponseEntity<String>  crearElemento(@RequestBody
Elemento elemento) {
        // Lógica para crear el elemento
    }

    @GetMapping("/elementos")
    public ResponseEntity<List<Elemento>> obtenerElementos() {
        // Lógica para obtener elementos
    }
}
```

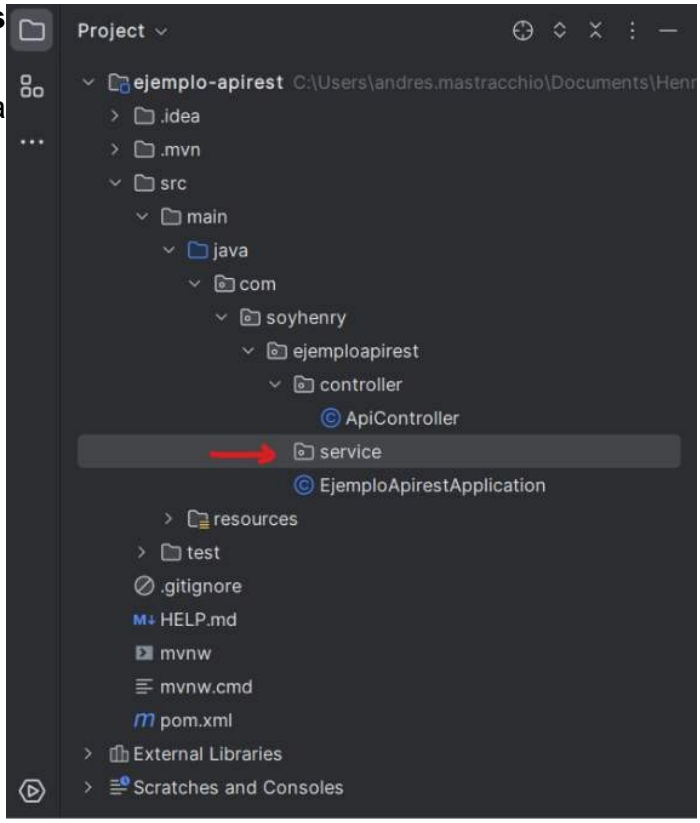
La anotación @RequestBody indica que el elemento que recibirá el endpoint como argumento es un objeto de tipo JSON. Esto es, cuando realizamos la petición HTTP, pasaremos en el body un objeto con la propiedad del elemento:

```
{
     “nombre”: “nombre de ejemplo”
}
```

Si en lugar de dicha anotación, usáramos @RequestParam, entonces el valor debería ser pasado como un parametro de la url de la request:
http://localhost:8080/api/v1/elemento/crear?nombre=nombre de ejemplo
Ten en cuenta que la elección de cuál forma utilizar dependerá de los requerimientos del negocio y seguridad en cuanto a la calidad y extensión de la información que debe viajar al endpoint y los recursos que el mismo disponibilizaría.

Paso 4: Desarrollar servicios con interfaz e implementación

Crea un paquete llamado "service", a la misma altura que el de controller.



Dentro de este paquete, crea una interfaz llamada "ElementoService" que defina los métodos para crear y obtener elementos.

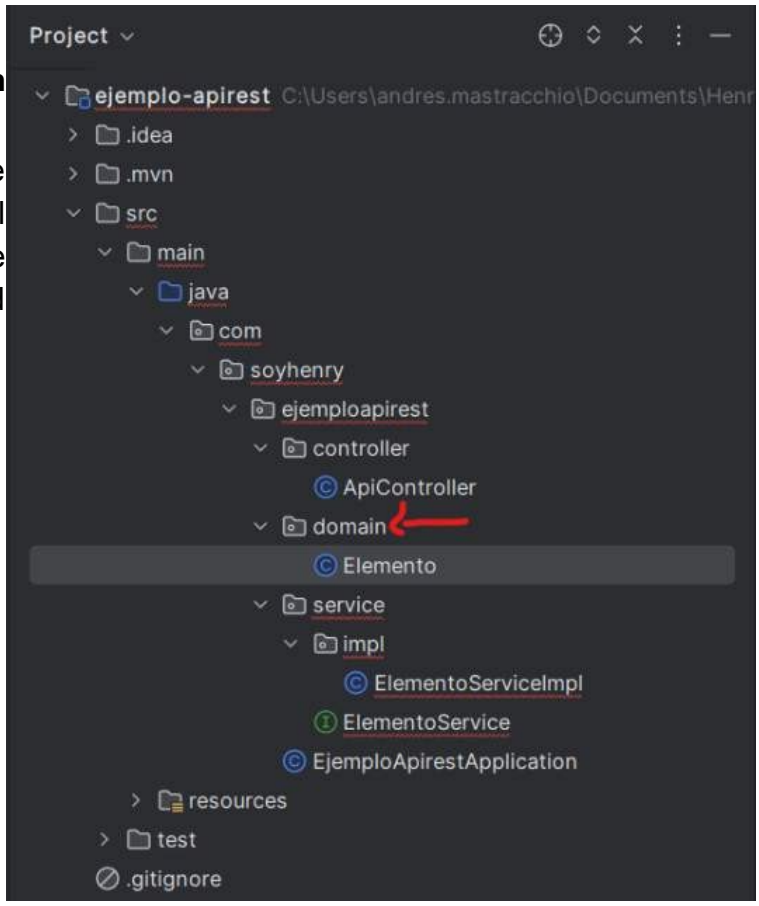
```
public interface ElementoService {
    Elemento crearElemento(Elemento elemento);
    List<Elemento> obtenerElementos();
}
```

Crea un nuevo paquete dentro del paquete service que se llame 'impl', donde crearas una clase llamada "ElementoServiceImpl" que implemente la interfaz "ElementoService". Debe estar anotada como @Component o @Service para que Spring la reconozca como un Bean y la gestione dentro del contexto. Aquí es donde implementarás la lógica para crear y obtener elementos. En este ejemplo, estaremos manipulando una lista de Elemento.

```
@Service
public class ElementoServiceImpl implements ElementoService {
    private List<Elemento> elementos = new ArrayList<>();

    @Override
    public Elemento crearElemento(Elemento elemento) {
        elementos.add(elemento);
        return elemento;
    }

    @Override
    public List<Elemento> obtenerElementos() {
        return elementos;
    }
}
```

Paso 5: Lógica de negocio con lista en memoria

Crea un paquete (a la misma altura que controller y service) llamado entity, model o domain y dentro genera una clase "Elemento" que representará la entidad con la que trabajarás en la API.




```
public class Elemento {
    private String nombre;
```

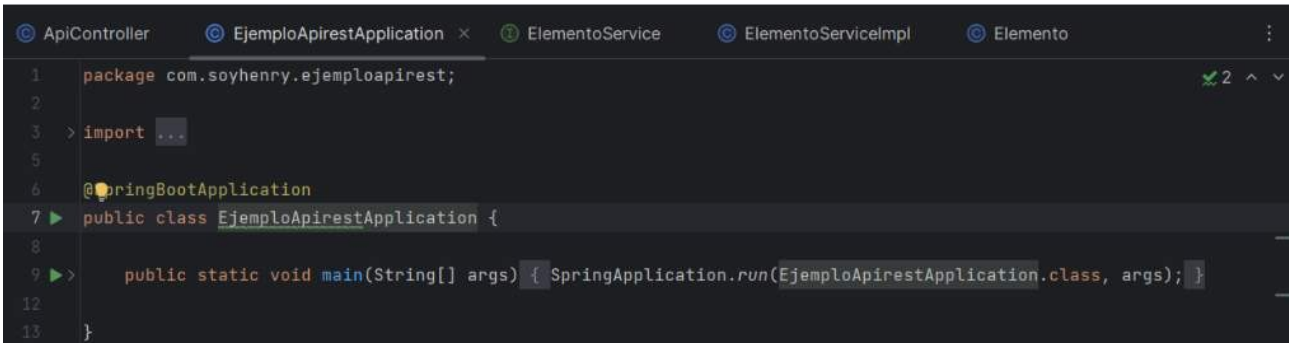
```
     // Puedes colocar otros atributos si deseas y no olvides que esta debe ser una clase POJO.
}
```

En el controlador, inyecta el servicio "ElementoService" y utilízalo para interactuar con la lógica de negocio.

```
@RestController
public class ApiController {
    private final ElementoService elementoService;

                // Esta es una inyección por constructor. Recuerda que la anotación @Autowired en este caso es opcional, ya que desde la versión 4.3 de Spring, cuando inyectamos por constructor (la forma recomendada por Spring) Spring entiende automáticamente la inyección.
    @Autowired
    public ApiController(ElementoService elementoService) {
        this.elementoService = elementoService;
    }

    @PostMapping("/crear")
       public  ResponseEntity<Elemento>  crearElemento(@RequestBody Elemento elemento) {
                                                                    return ResponseEntity.ok(elementoService.crearElemento(elemento));
    }

    @GetMapping("/elementos")
    public ResponseEntity<List<Elemento>> obtenerElementos() {
                                                                    return ResponseEntity.ok(elementoService.obtenerElementos());
    }
}
```

Paso 6: Ejecutando la aplicación

Para correr la aplicación, debemos ir a la clase que tiene el método main y ejecutar el método desde el botón verde de play.



Luego, podremos ver la consola para saber si la aplicación inició con éxito

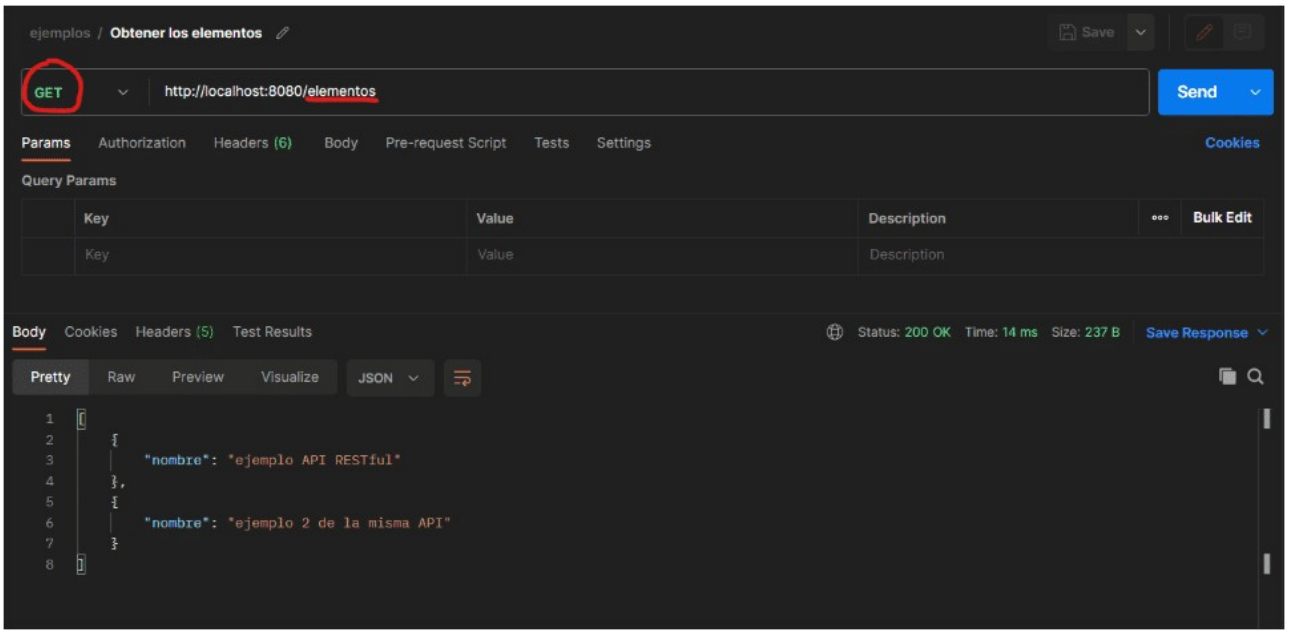
```
Run   EjemploApirestApplication

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.1.3)

2023-09-01T09:02:26.681-03:00  INFO 23380 --- [  restartedMain] c.s.e.EjemploApirestApplication         : Starting EjemploApirestApplication using Java 17.0.7 with PID
 23380 (C:\Users\andres.mastracchio\Documents\Henry\WKS-Spring\Clase-6\ejemplo-apirest\target\classes started by andres.mastracchio in C:\Users\andres
 .mastracchio\Documents\Henry\WKS-Spring\Clase-6\ejemplo-apirest)
2023-09-01T09:02:26.689-03:00  INFO 23380 --- [  restartedMain] c.s.e.EjemploApirestApplication         : No active profile set, falling back to 1 default profile: "default"
2023-09-01T09:02:26.790-03:00  INFO 23380 --- [  restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools
 .add-properties' to 'false' to disable
2023-09-01T09:02:26.790-03:00  INFO 23380 --- [  restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging
 .level.web' property to 'DEBUG'
2023-09-01T09:02:28.429-03:00  INFO 23380 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http)
2023-09-01T09:02:28.442-03:00  INFO 23380 --- [  restartedMain] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2023-09-01T09:02:28.442-03:00  INFO 23380 --- [  restartedMain] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.12]
2023-09-01T09:02:28.523-03:00  INFO 23380 --- [  restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2023-09-01T09:02:28.528-03:00  INFO 23380 --- [  restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1732 ms
2023-09-01T09:02:29.107-03:00  INFO 23380 --- [  restartedMain] o.s.b.d.a.OptionalLiveReloadServer       : LiveReload server is running on port 35729
2023-09-01T09:02:29.187-03:00  INFO 23380 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http) with context path ''
2023-09-01T09:02:29.198-03:00  INFO 23380 --- [  restartedMain] c.s.e.EjemploApirestApplication         : Started EjemploApirestApplication in 3.072 seconds (process
 running for 4.122)

emplo-apirest > src > main > java > com > soyhenry > ejemploapirest > EjemploApirestApplication   LF   UTF-8   Tab*
```

Para realizar una prueba rápida, podemos crear dos requests en Postman y verificar el funcionamiento. Cabe recordar, que debemos hacer las peticiones como JSON para este ejemplo. La url del endpoint la sacamos del valor definido en el @PostMapping y @GetMapping respectivamente.

PETICIÓN POST - CREAR UN ELEMENTO

```
ejemplos / Crear Elemento                                   Save

POST   http://localhost:8080/crear                          Send

Params   Authorization   Headers (8)   Body   Pre-request Script   Tests   Settings   Cookies

none   form-data   x-www-form-urlencoded   raw   binary   GraphQL   JSON   Beautify

1 {
2   "nombre": "ejemplo API RESTful"
3 }

Body   Cookies   Headers (5)   Test Results   Status: 200 OK   Time: 104 ms   Size: 196 B   Save Response

Pretty   Raw   Preview   Visualize   JSON

1 {
2   "nombre": "ejemplo API RESTful"
3 }
```

PETICIÓN GET - OBTENER LOS ELEMENTO



¡Listo, has creado una API RESTful con Spring Boot!

Recuerda que este es solo un ejemplo básico. Puede ser expandido de mil maneras con los proyectos y soluciones de Spring, utilizando Spring Data para conectar a una base de datos o Spring Security para colocar una capa de seguridad.

Puedes realizar otro ejemplo, un poco más elaborado con base de datos, mirando los siguientes tutoriales:

https://youtu.be/hzEpwm_Ew7I

https://youtu.be/uhRdxB6TRuQ